

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

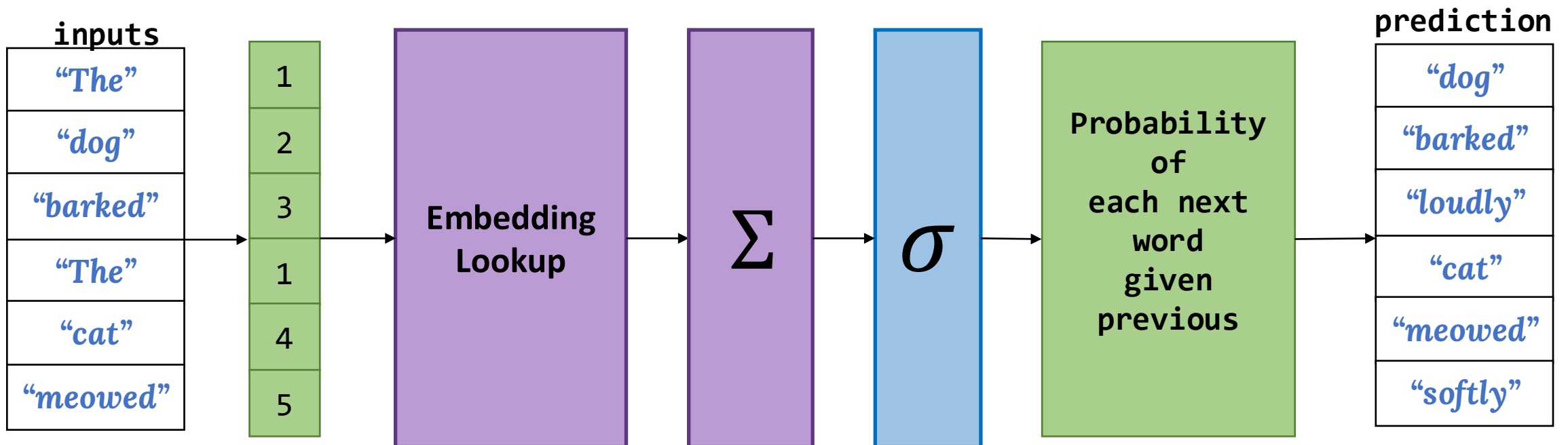
April 16, 2026
Thursday



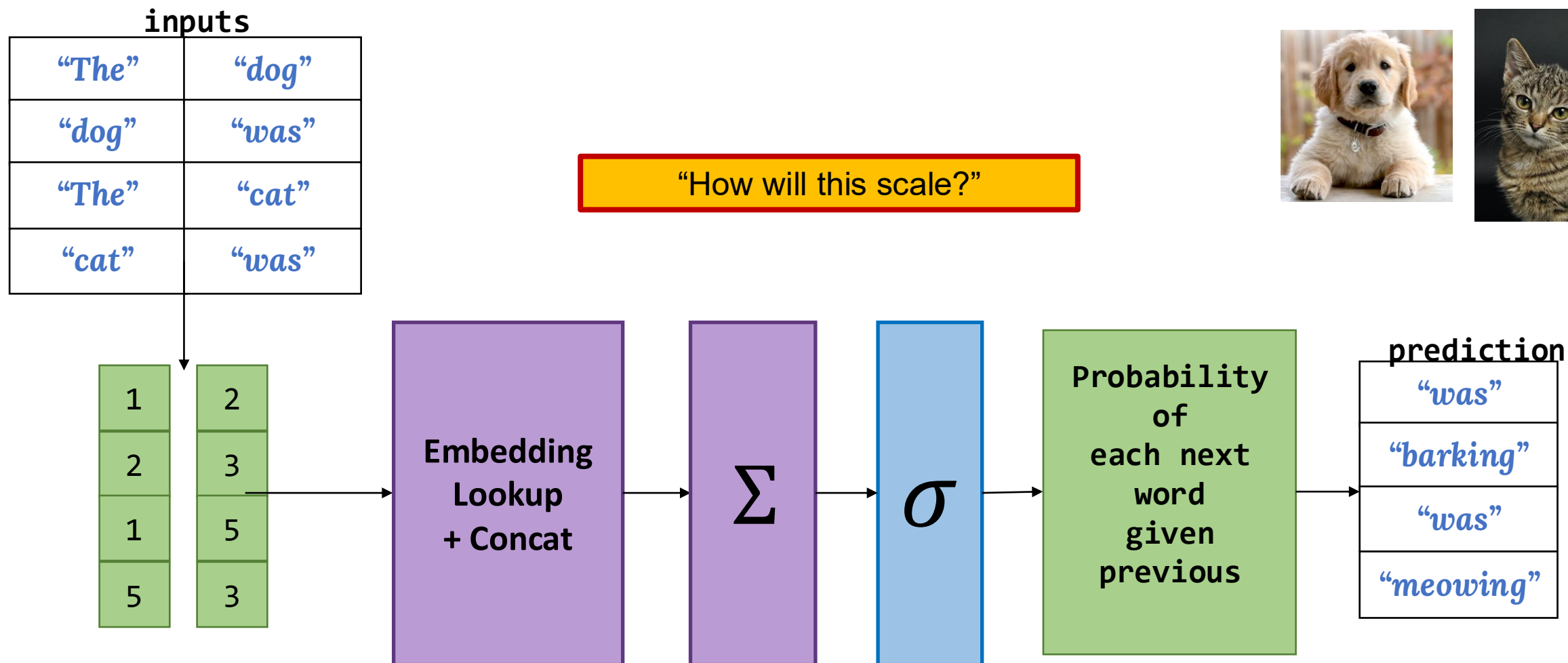
Today's goal – learn about RNNs and Transformers

- (1) Recurrent Neural Network (RNN)
- (2) Sequence-to-sequence modeling
- (3) Transformers
- (4) Class activity: Self-attention as matrix operation

Review: Bigram Language Model Architecture



Review: Complete Trigram Language Model

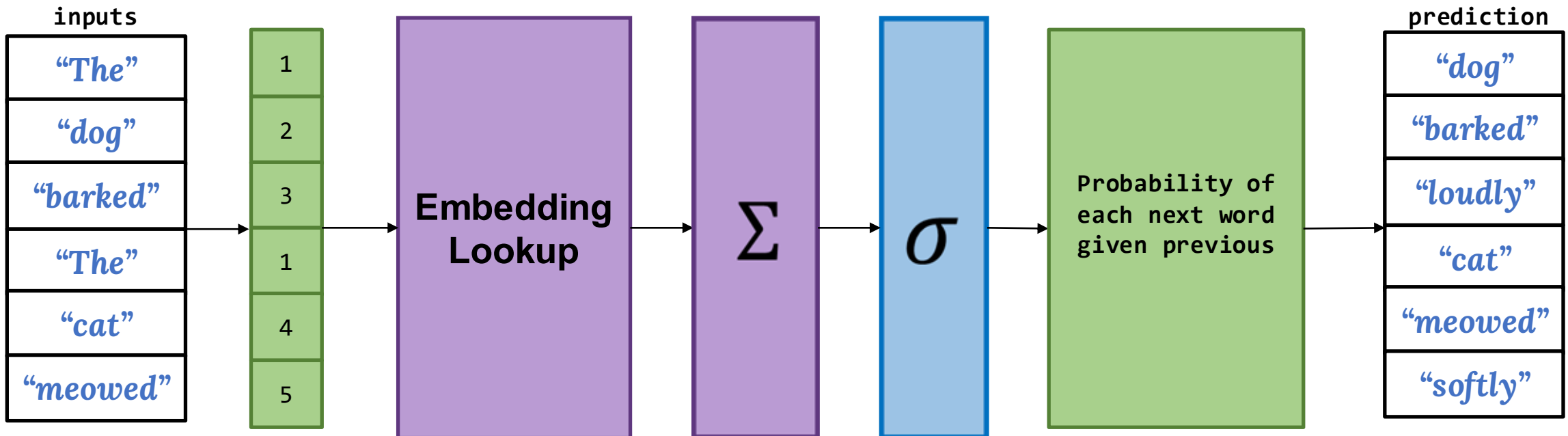


Limitations of the N-gram model

What problems do we run into using Feed Forward N-gram models?

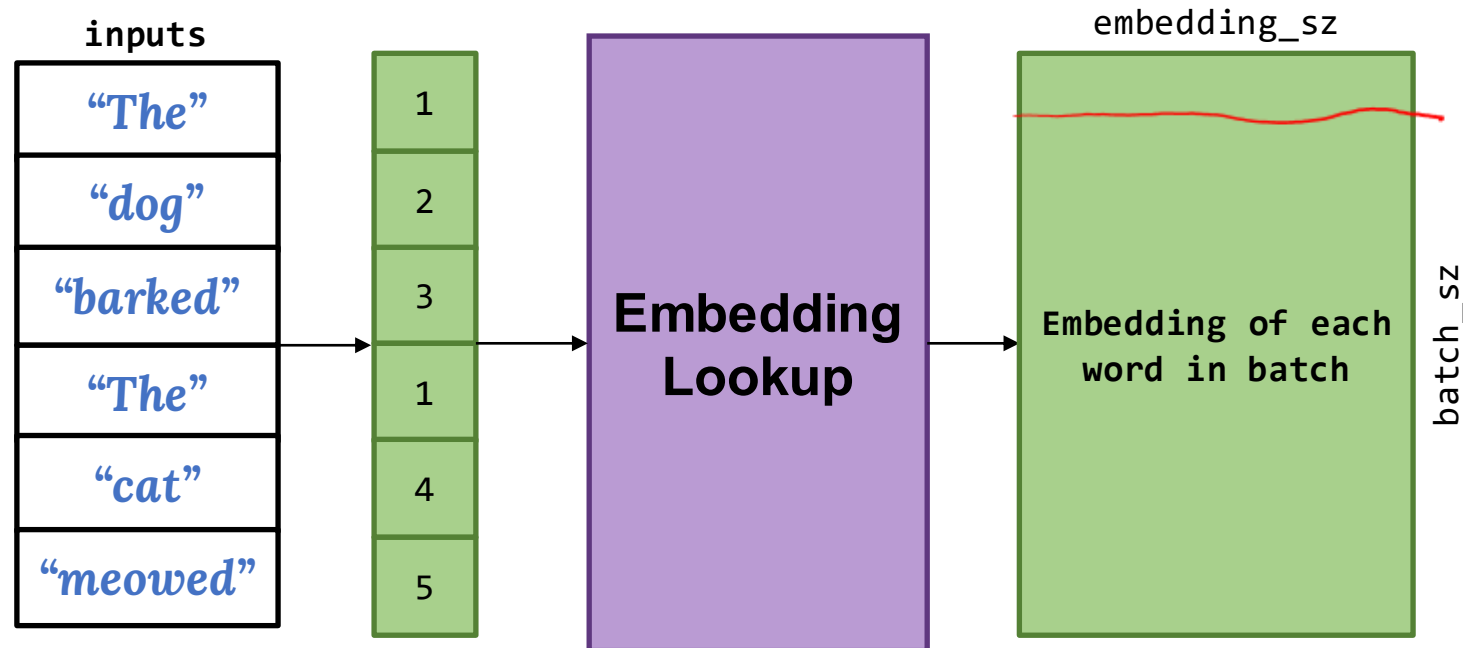
Size of Feed Forward bigram Model

Let's look at bigram model and count the number of weights.



Size of Feed Forward bigram Model

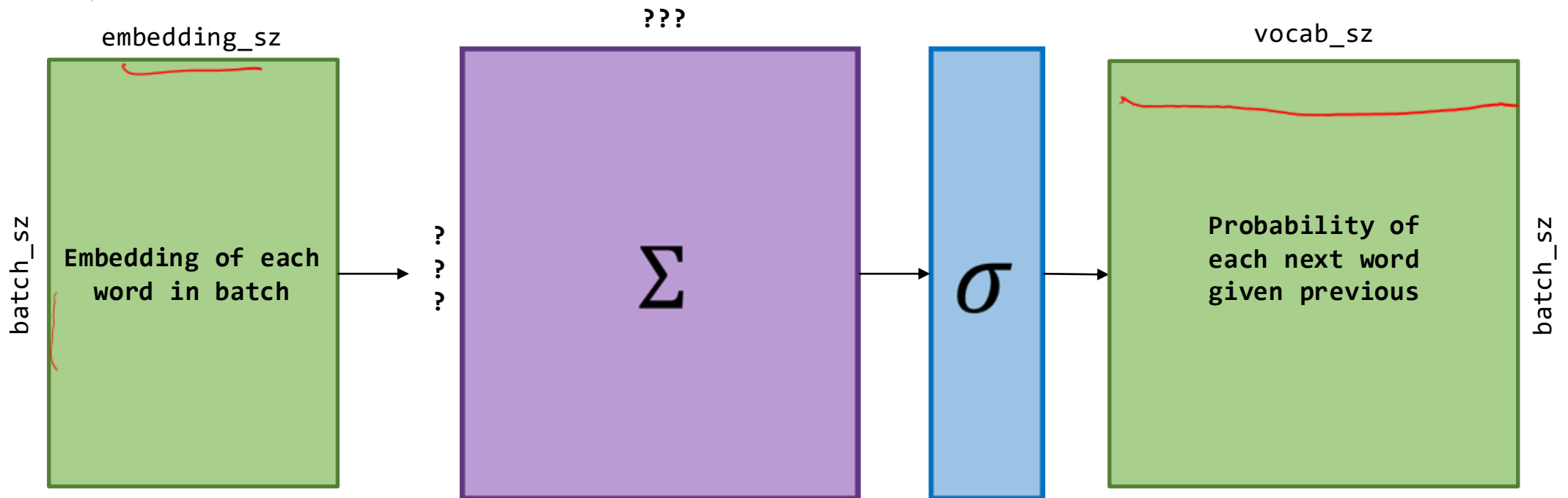
To preform embedding lookup on our entire batch, we just need one embedding matrix of size: (vocab_sz, embedding_sz)



Size of Feed Forward bigram Model

What size do we need the linear layer to be in order to map:

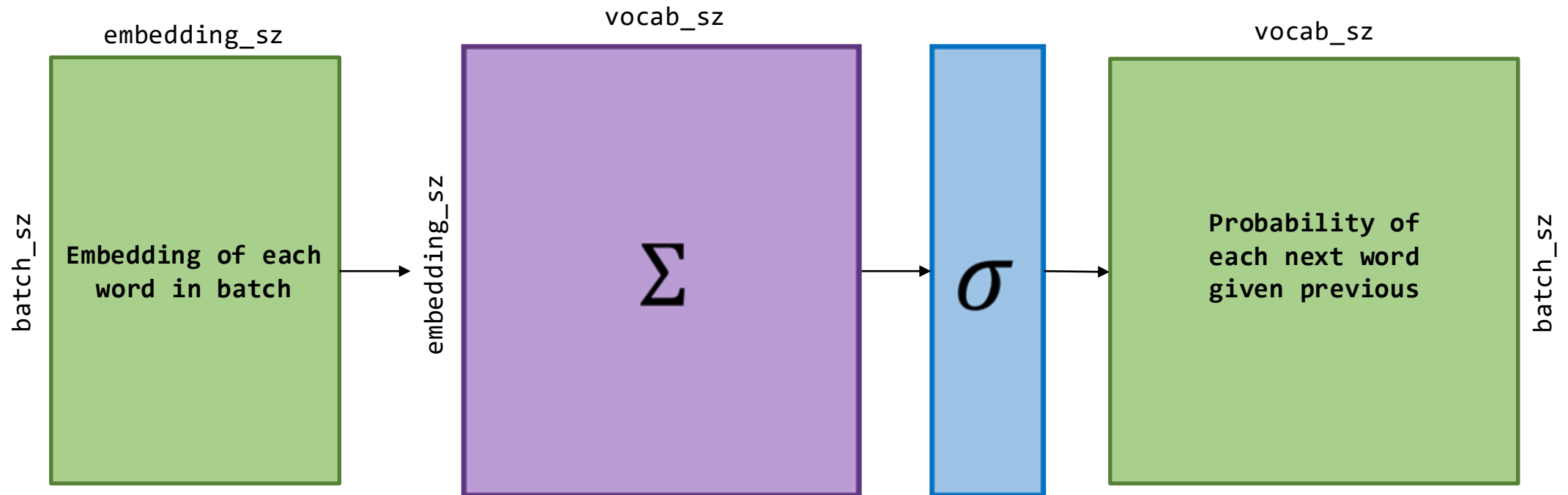
$$(\text{batch_sz}, \text{embedding_sz}) \times (???, ???) \rightarrow (\text{batch_sz}, \text{vocab_sz})$$



Size of Feed Forward bigram Model

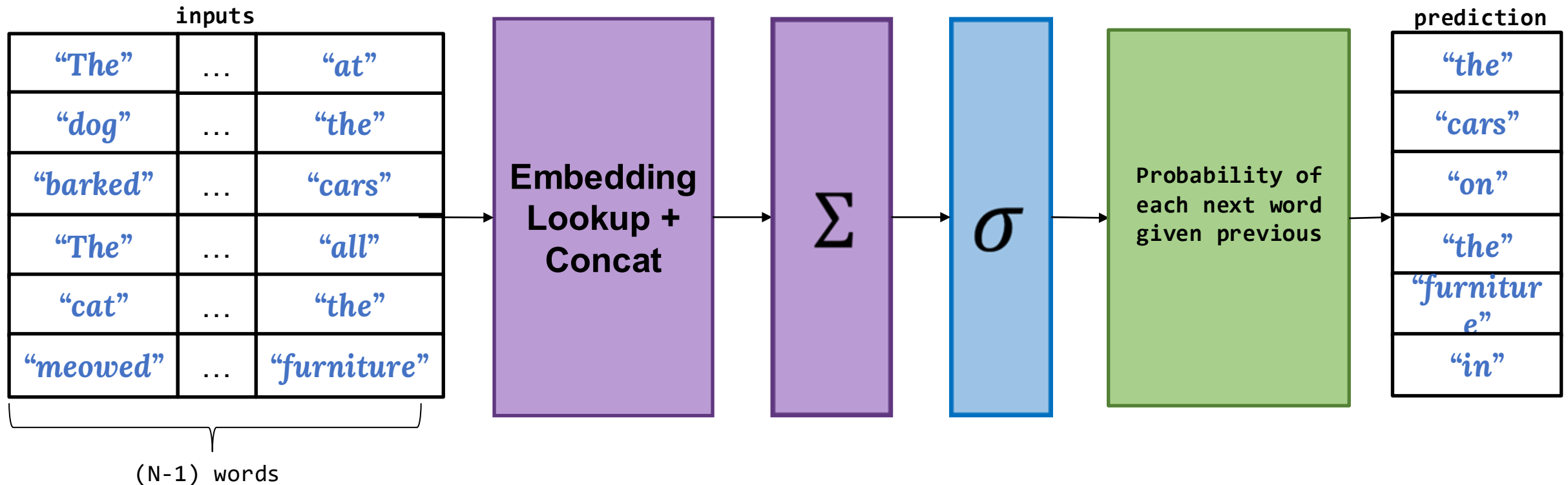
What size do we need the linear layer to be in order to map:

$$(\text{batch_sz}, \text{embedding_sz}) \times (???, ???) \rightarrow (\text{batch_sz}, \text{vocab_sz})$$



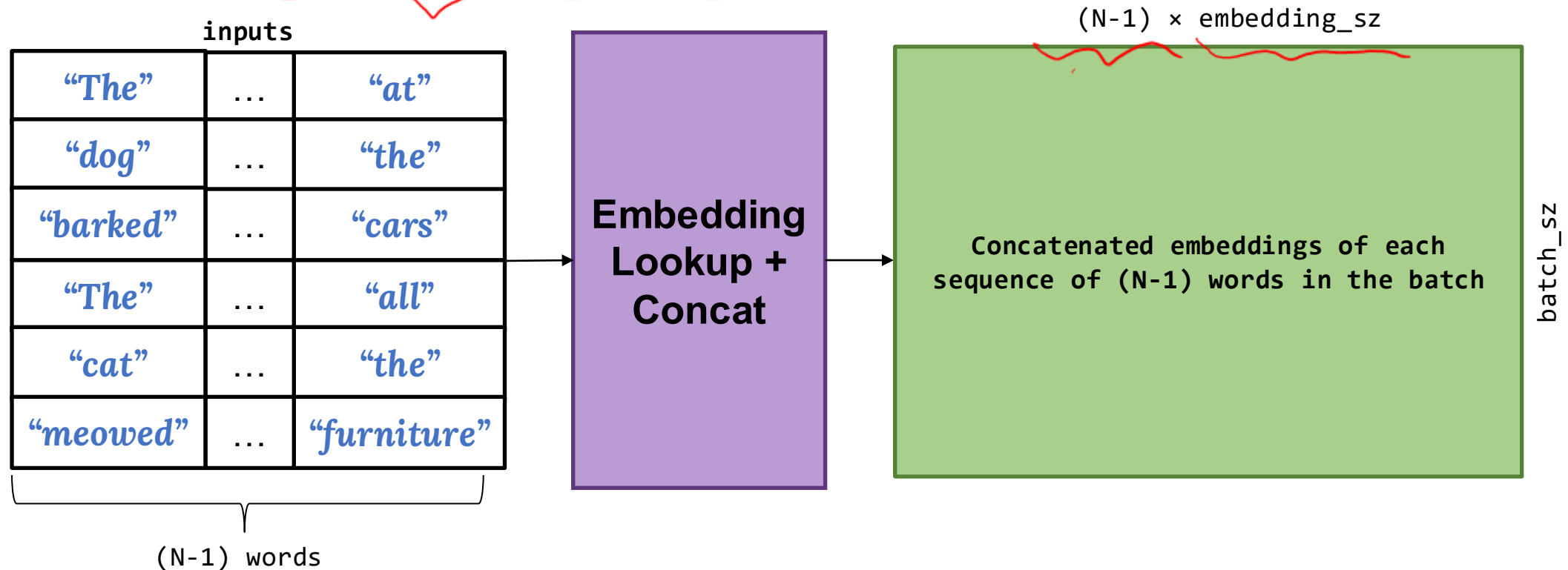
Size of Feed Forward N-gram Model

So what happens in the N-gram case?



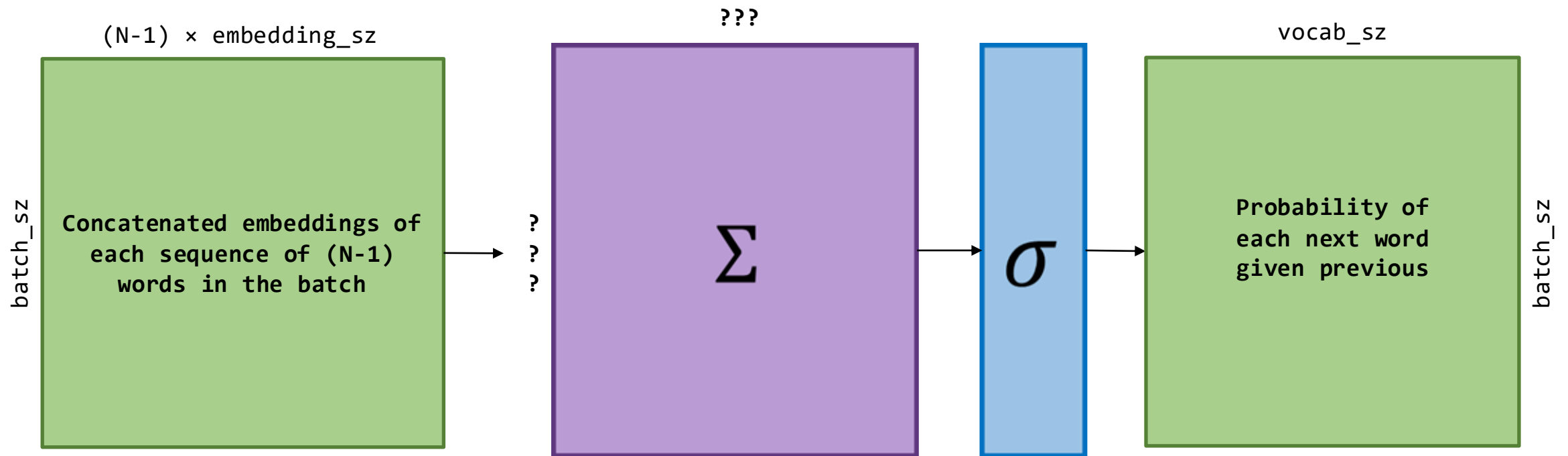
Size of Feed Forward N-gram Model

Embedding lookup + Concatenation still requires only one embedding matrix of size: (vocab_sz, embedding_sz)



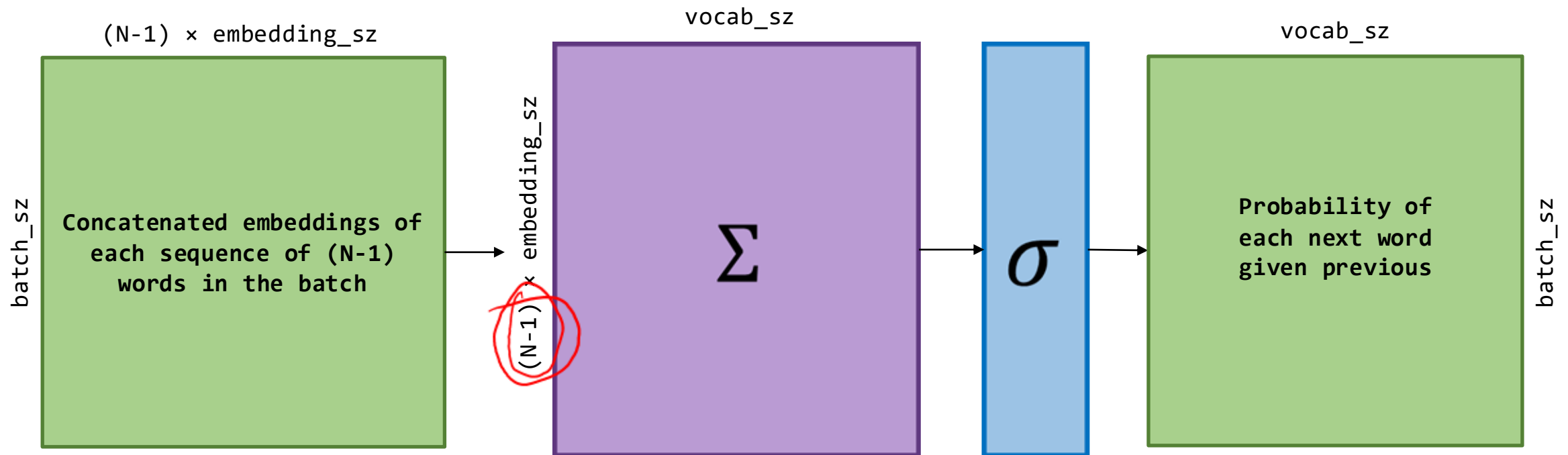
Size of Feed Forward N-gram Model

But what happens to our feed forward layer?



Size of Feed Forward N-gram Model

It needs to be size: $((N-1) \times \text{embedding_sz}, \text{vocab_sz})$
For every word, we add $(\text{embedding_sz} \times \text{vocab_sz})$ more weights!



Limitations of the N-gram model

What problems do we run into using Feed Forward N-gram models?

1. As the size of **N** increases, the number of weights needed for the linear layer becomes far too large.

Limitations of the N-gram model

What problems do we run into using Feed Forward N-gram models?

1. As the size of **N** increases, the number of weights needed for the linear layer becomes far too large.
2. Using a fixed **N** creates problems with the flexibility of our model.



Limitations of the N-gram model

What problems do we run into using Feed Forward N-gram models?

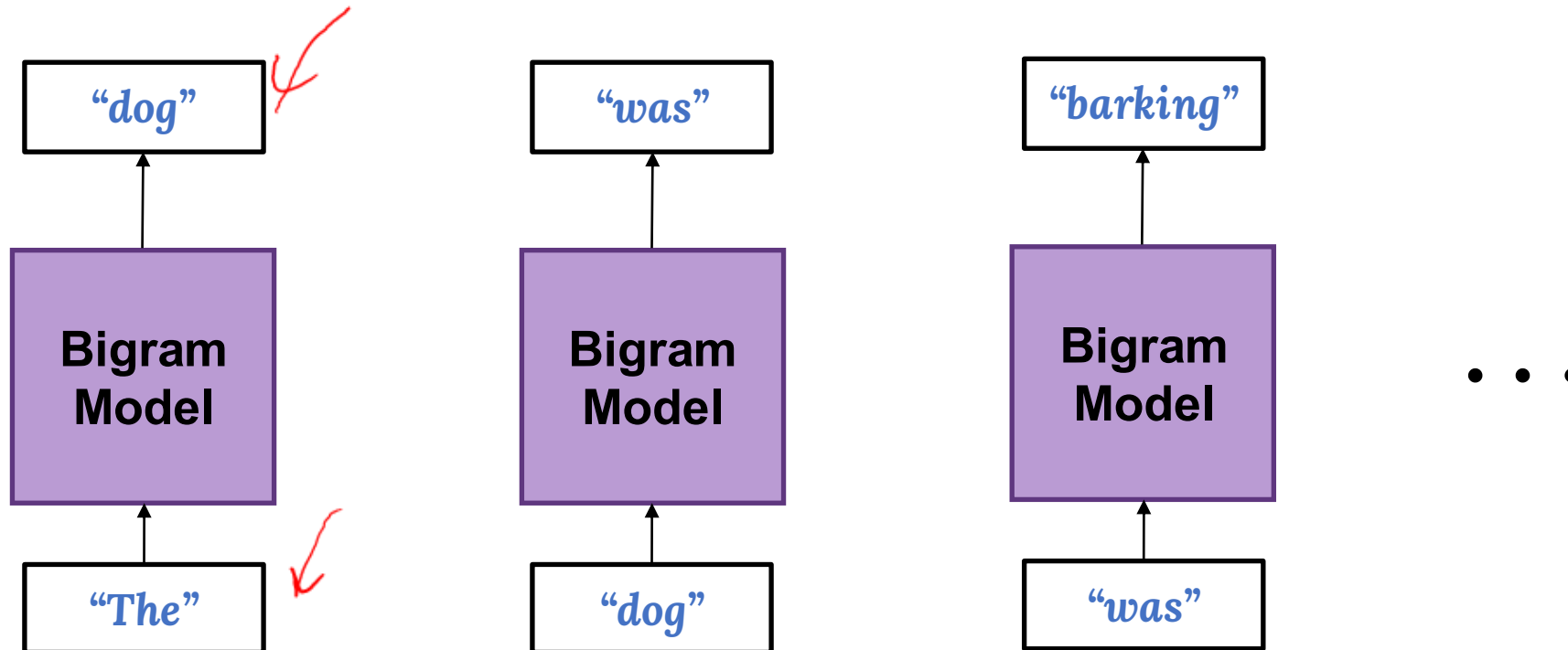
1. As the size of **N** increases, the number of weights needed for the linear layer becomes far too large.
1. Using a fixed **N** creates problems with the flexibility of our model.

We need a solution that is both **computationally cheap and more dynamic in terms of its memory of previously seen words.**

Recurrent Neural Network (RNN)

New Approach

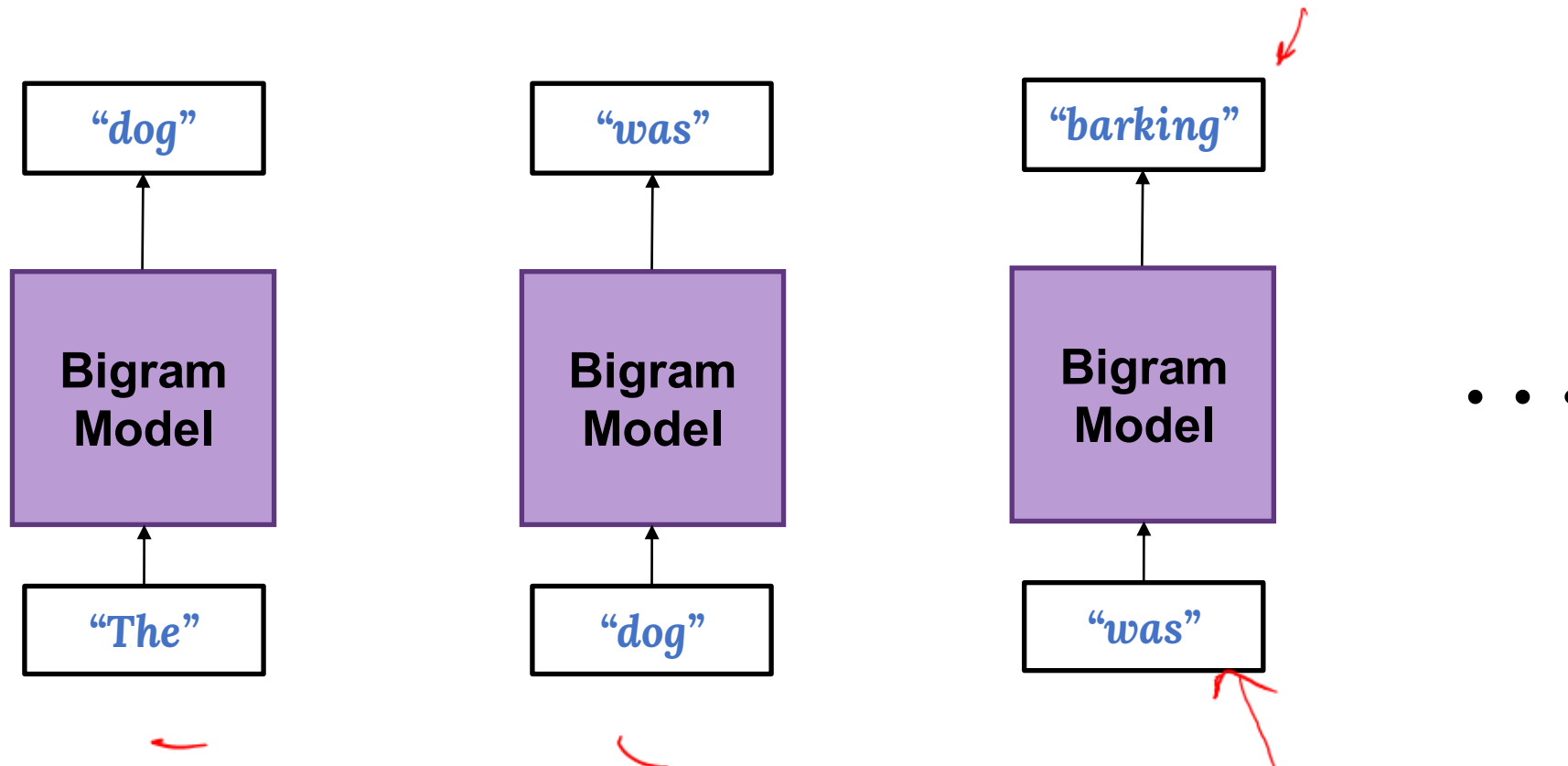
Let's revisit the bigram model and see several iterations of prediction using a bigram model:



New Approach

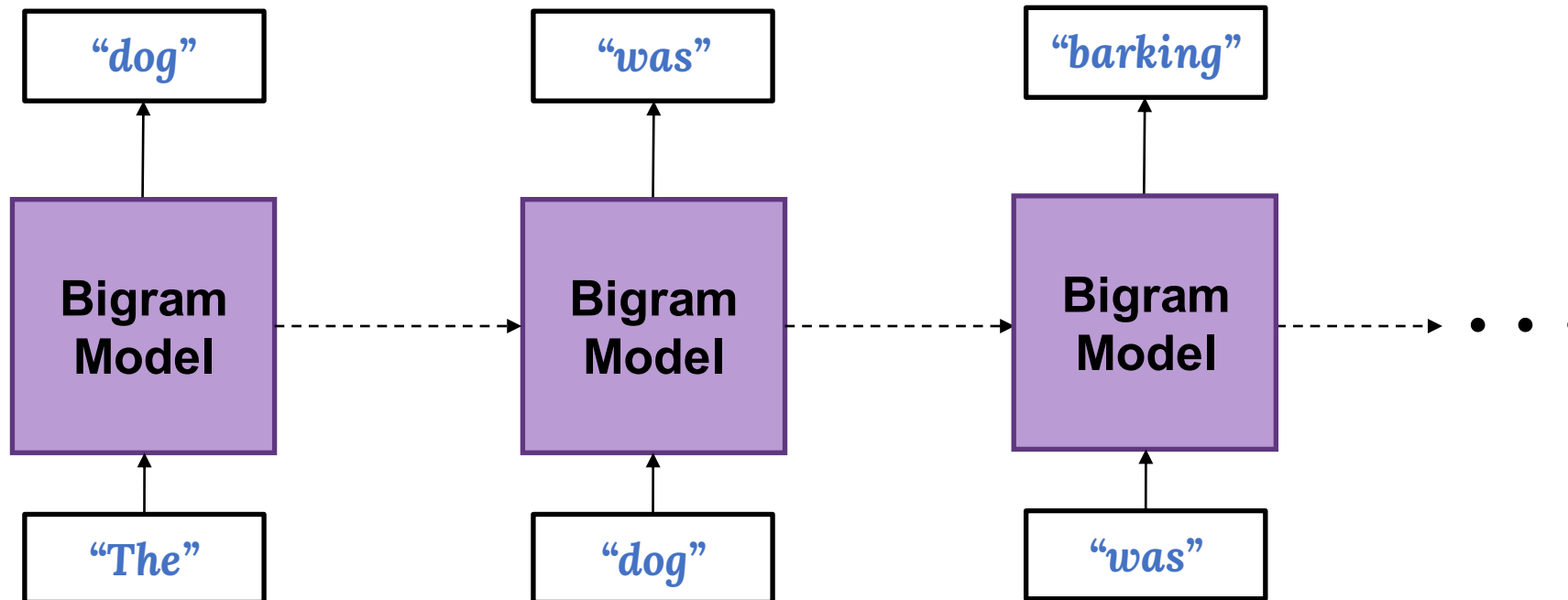
Any ideas?

Ideally, we would like to be able to keep “memory” of what words occurred in the past.



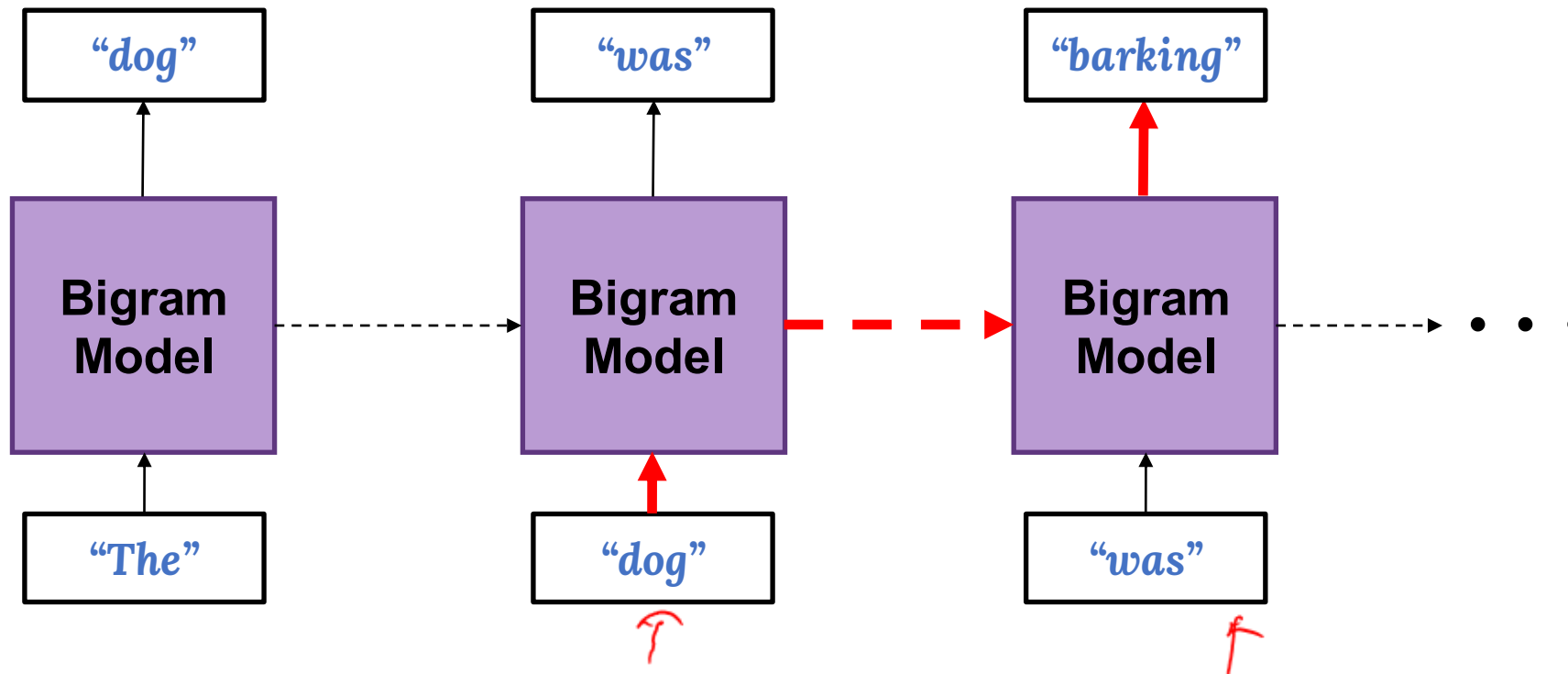
New Approach

What if we sequentially passed information from our previous bigram block into our next block?



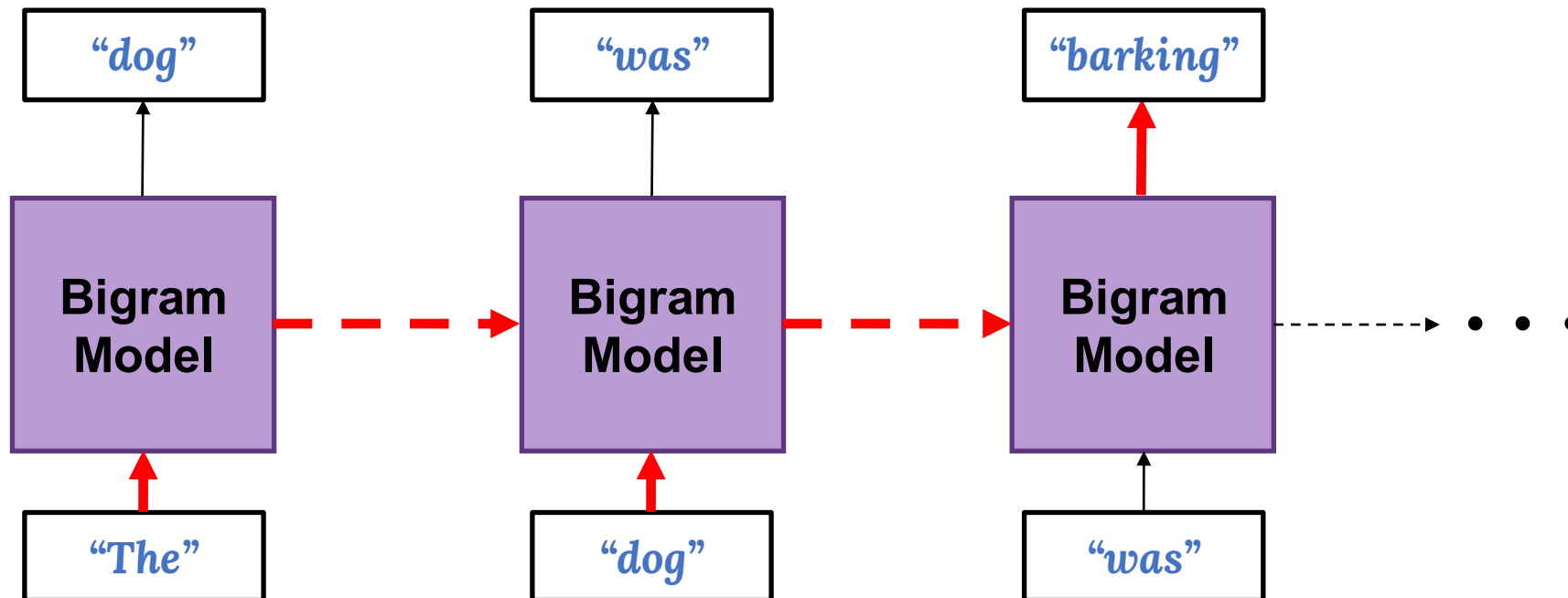
New Approach

If we follow the information flow, we see that when predicting “barking”, we have some way of knowing that “dog” was previously observed:



New Approach

In fact, we even have a way of knowing that “The” was observed!

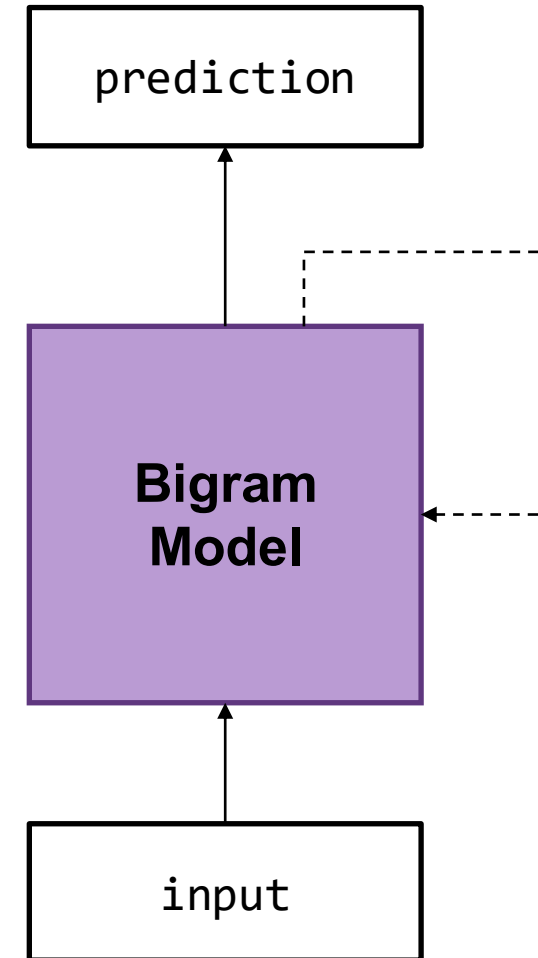


New Approach

We can represent this relationship using only one bigram block and connection that feeds from the output of the model back into the input.

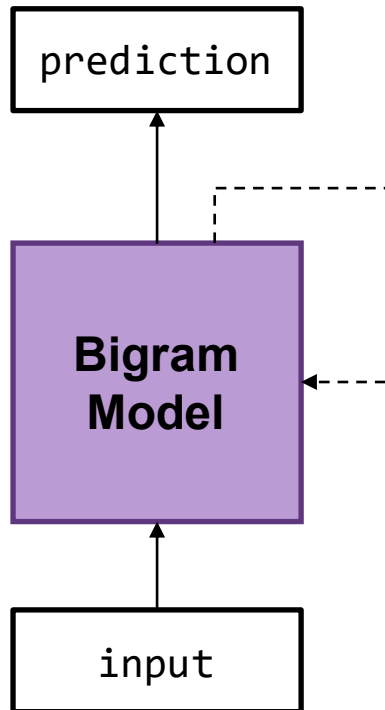
We call this connection a *recurrent* connection.

We call the previous representation the “unrolled” representation.

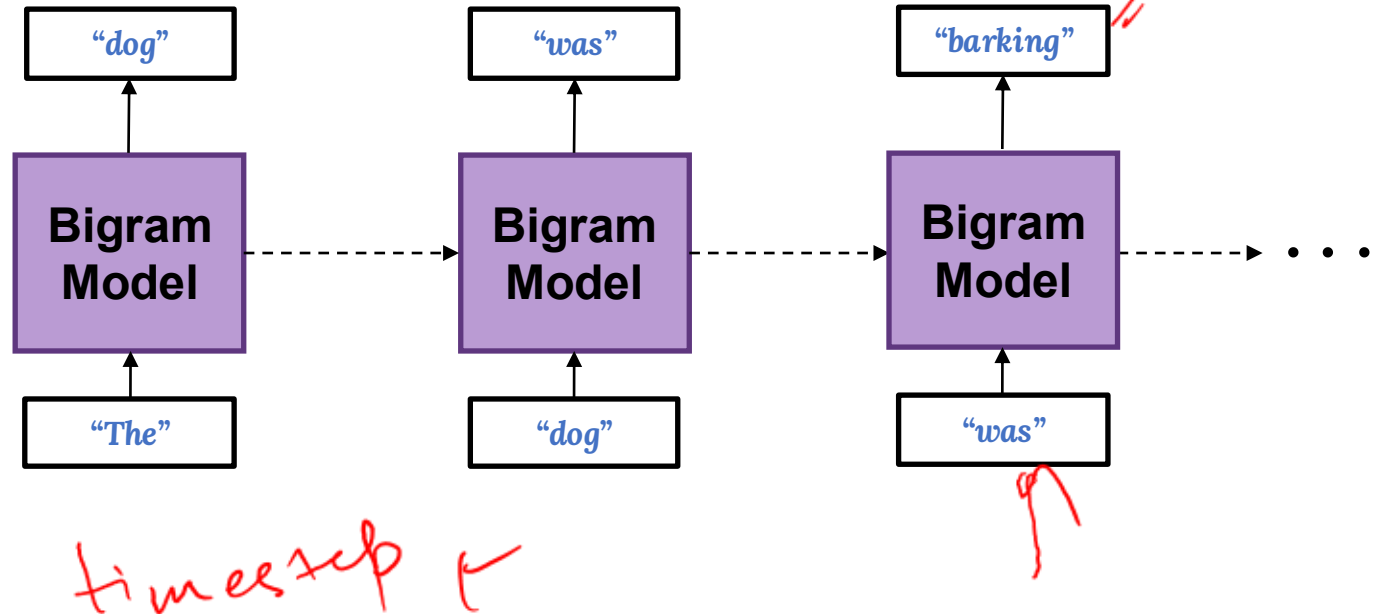


Different views of recurrent models

Recurrent view



Unrolled view



Recurrent Neural Network (RNN)

Recurrent Neural Networks are networks in the form of a directed *cyclic* graph.

They pass previous *state* information from previous computations to the next.

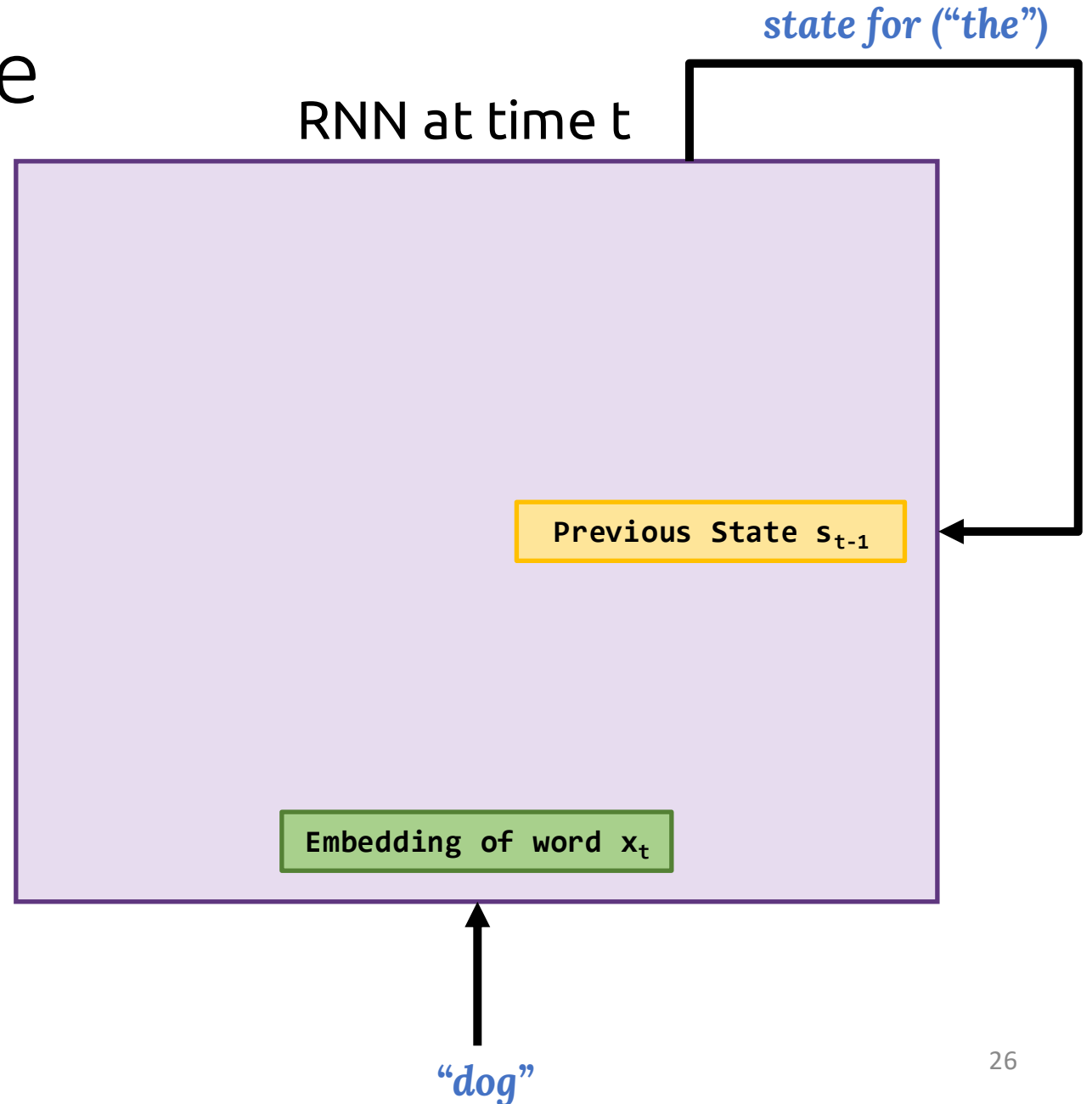
They can be used to process sequence data with relatively low model complexity when compared to feed forward models.

The block of computation that feeds its own output into its input is called the *RNN cell*.

Let's see how we can build one!

RNN Cell Architecture

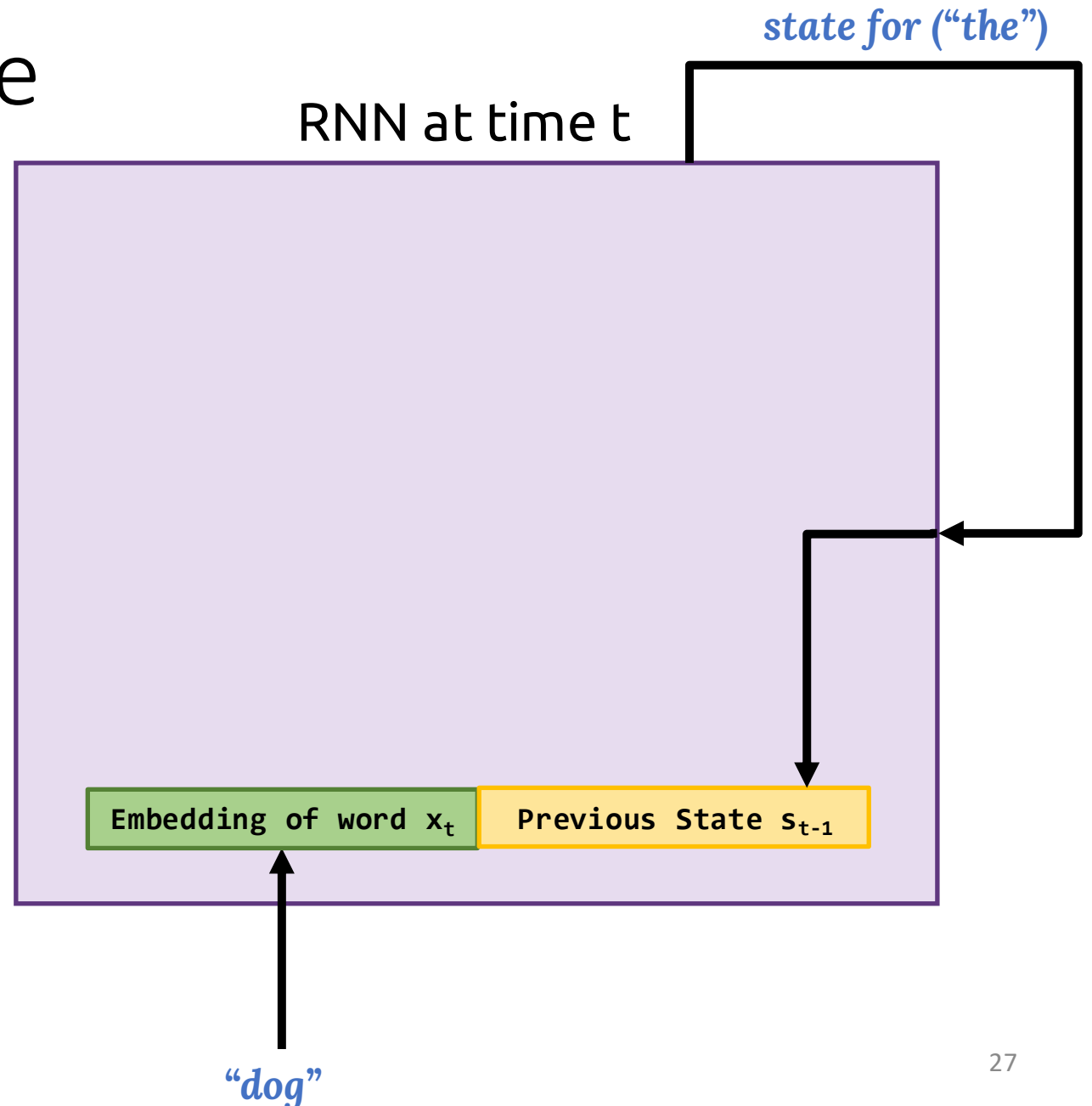
At each step of our RNN, we will get an input word, and a state vector from the previous cell.



RNN Cell Architecture

At each step of our RNN, we will get an input word, and a state vector from the previous cell.

We then concatenate the embedding and state vectors.

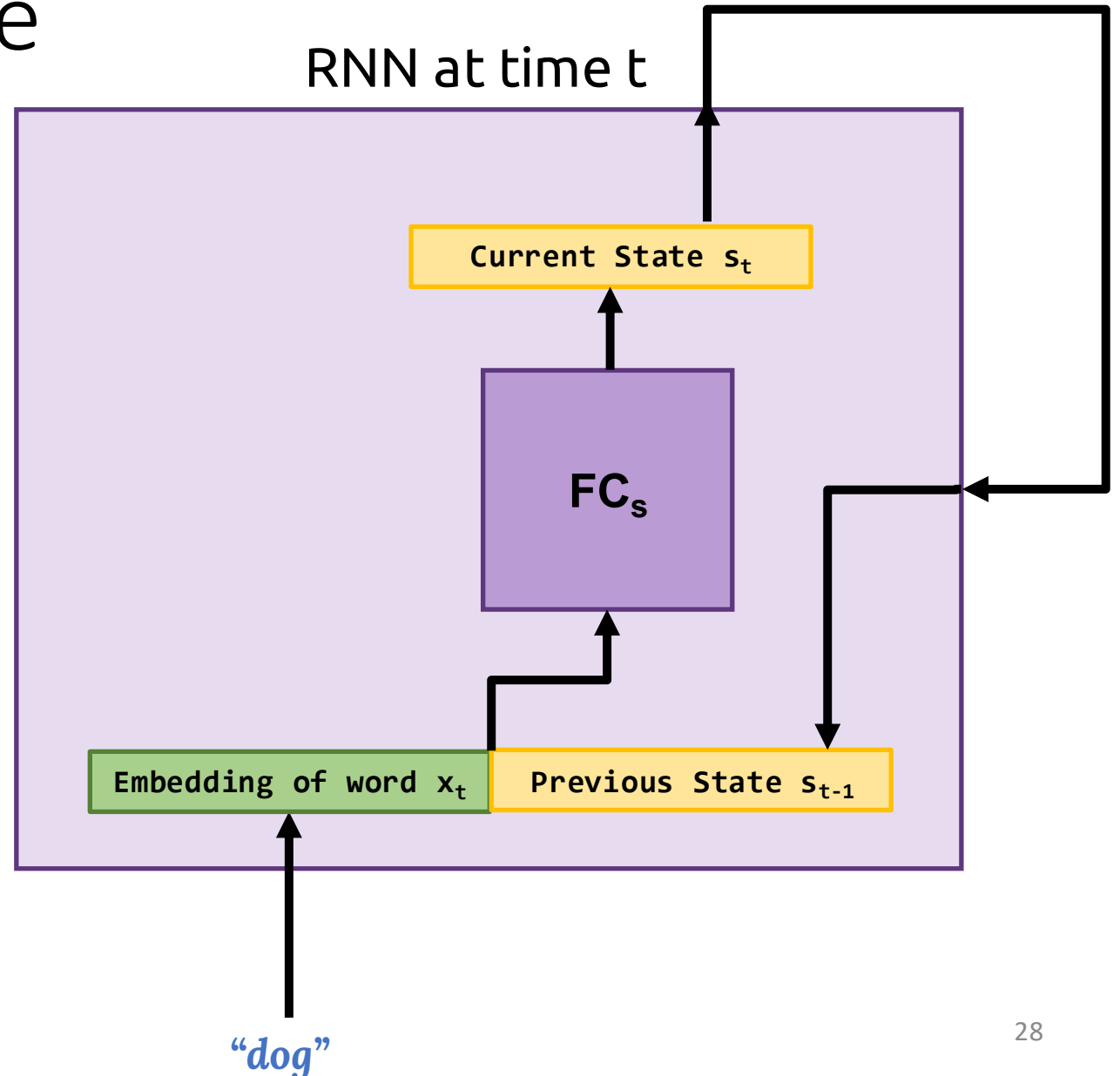


RNN Cell Architecture

At each step of our RNN, we will get an input word, and a state vector from the previous cell.

We then concatenate the embedding and state vectors.

We use a fully connected layer to compute the next state



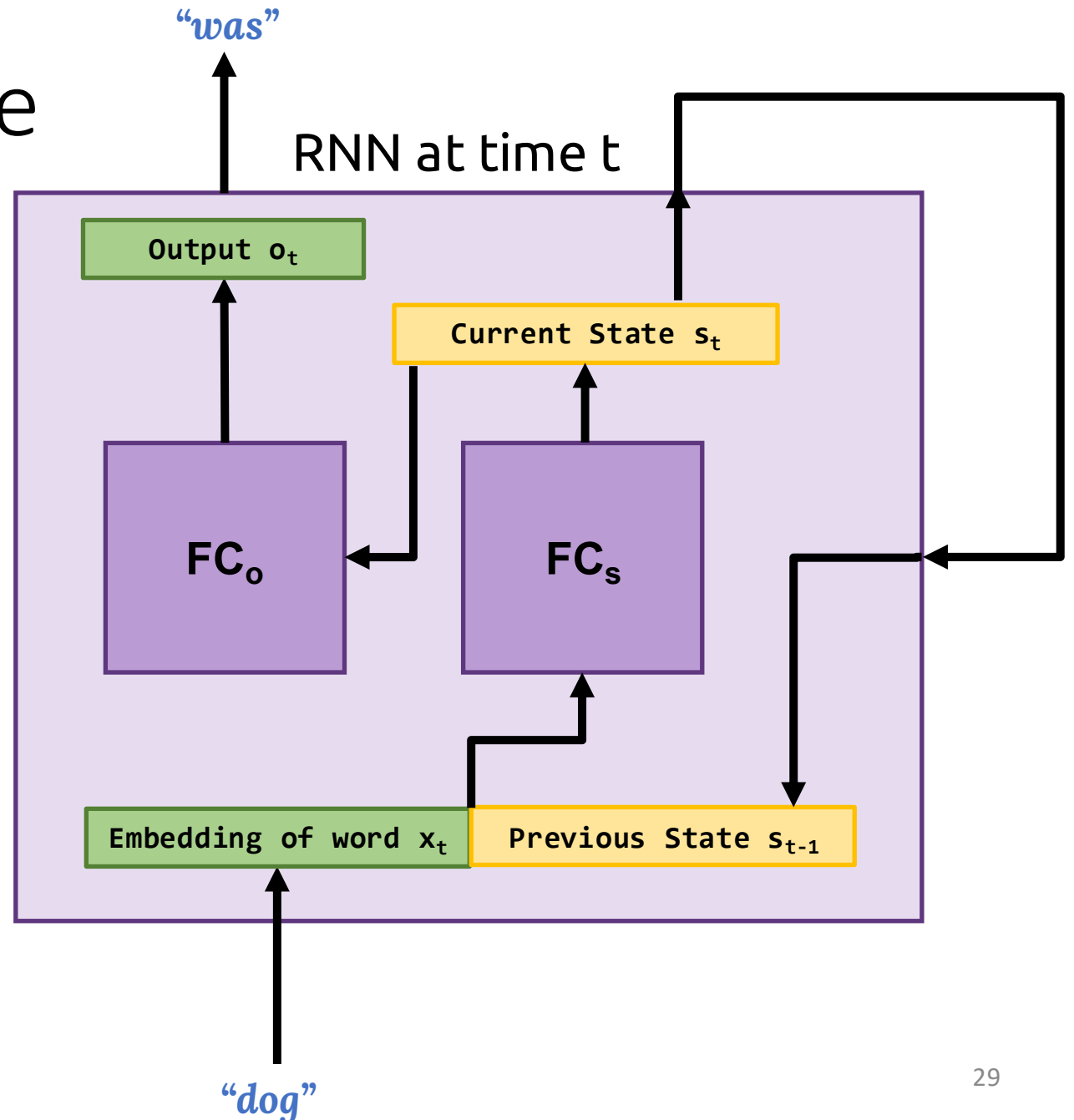
RNN Cell Architecture

At each step of our RNN, we will get an input word, and a state vector from the previous cell.

We then concatenate the embedding and state vectors.

We use a fully connected layer to compute the next state

We use another connected layer to get the output.

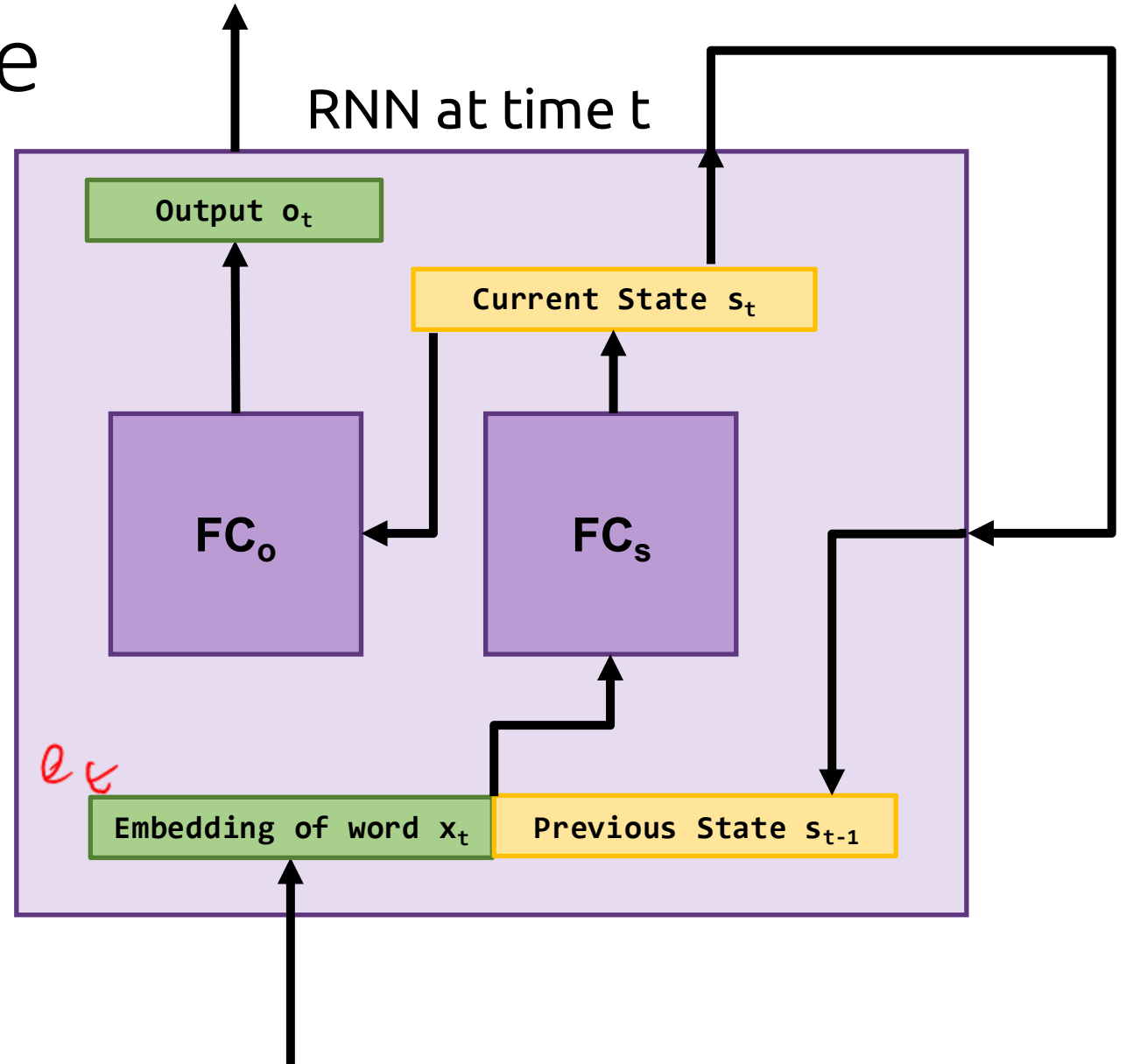


RNN Cell Architecture

We can represent the RNN in with the following equations:

$$\underline{s_t} = \rho(\underline{(e_t, s_{t-1})} W_r + b_r)$$

$$o_t = \sigma(s_t W_o + b_o)$$



RNN Cell Architecture

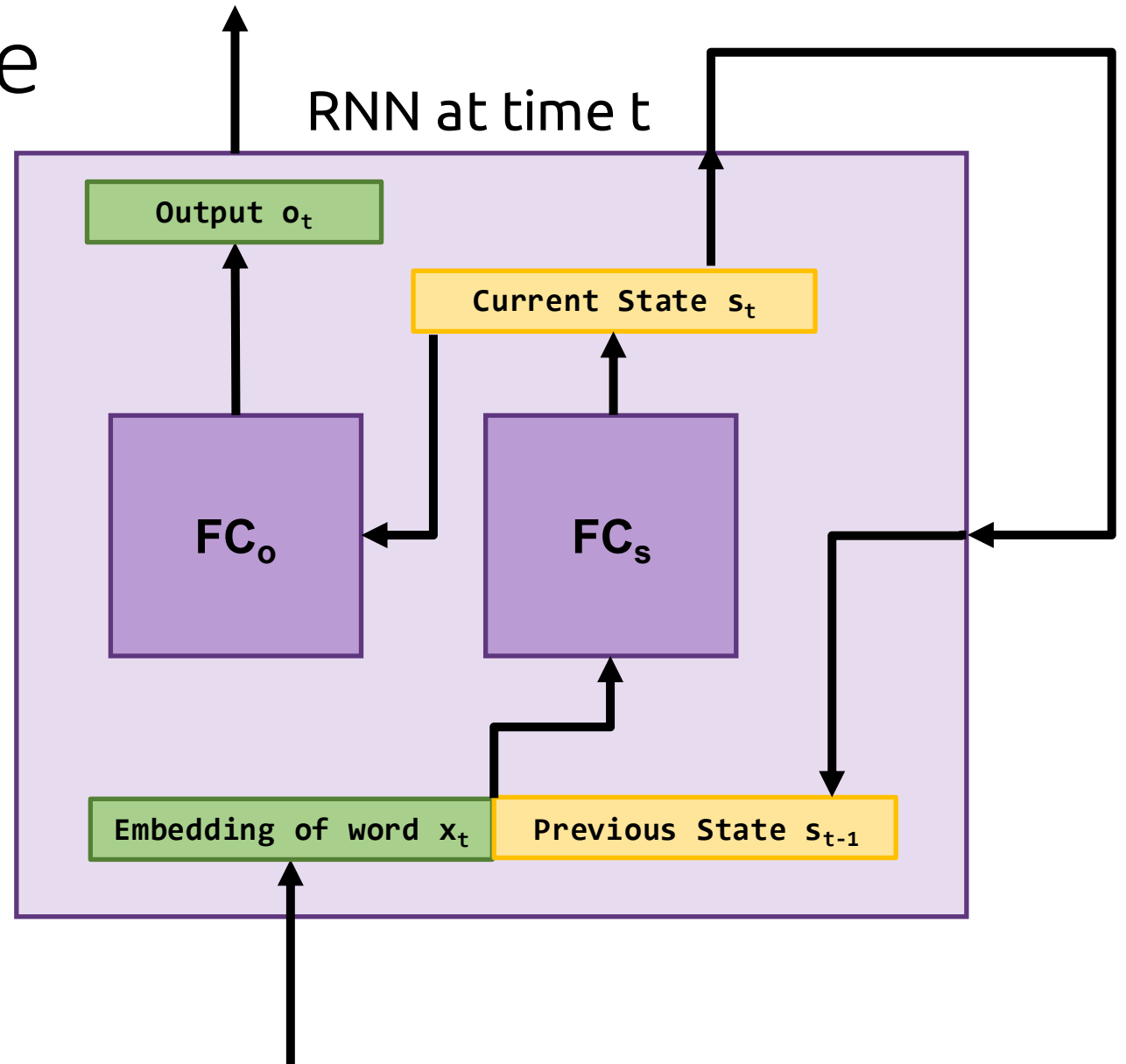
We can represent the RNN in with the following equations:

$$s_t = \rho((e_t, s_{t-1})W_r + b_r)$$

$$o_t = \sigma(s_t W_o + b_o)$$

**Nonlinear activations
(e.g. sigmoid, tanh)**

Any questions?



RNN Cell Architecture

We can represent the RNN in with the following equations:

$$s_t = \rho((e_t, s_{t-1})W_r + b_r)$$

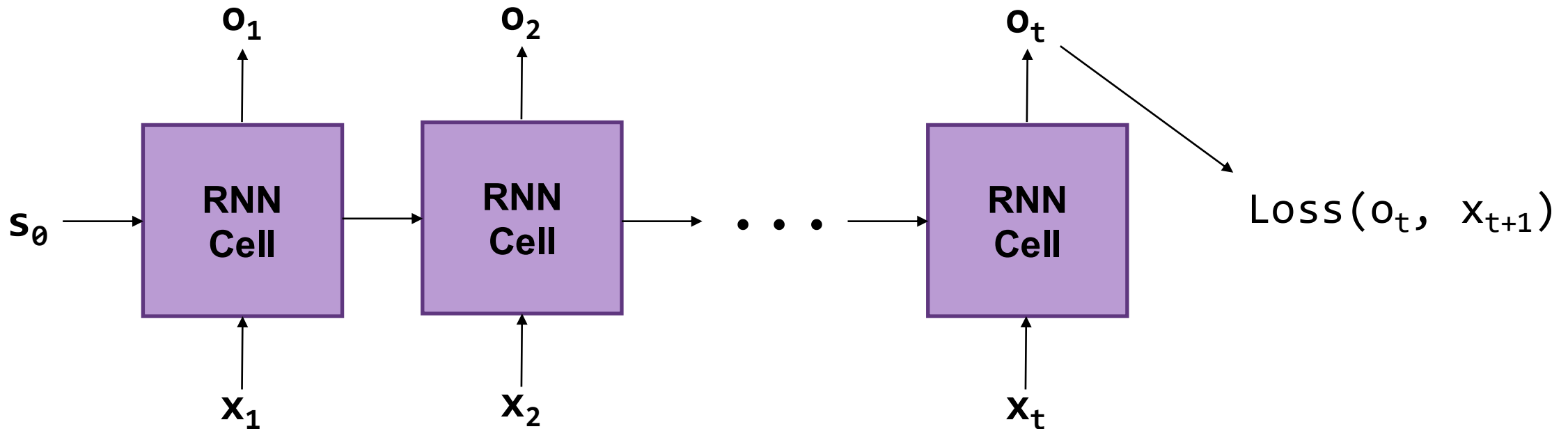
$$o_t = \sigma(s_t W_o + b_o)$$

This brings up an immediate question: **what is s_0 ?**

Typically, we initialize s_0 to be a vector of zeros (i.e. “initially, there is no memory of any previous words”)

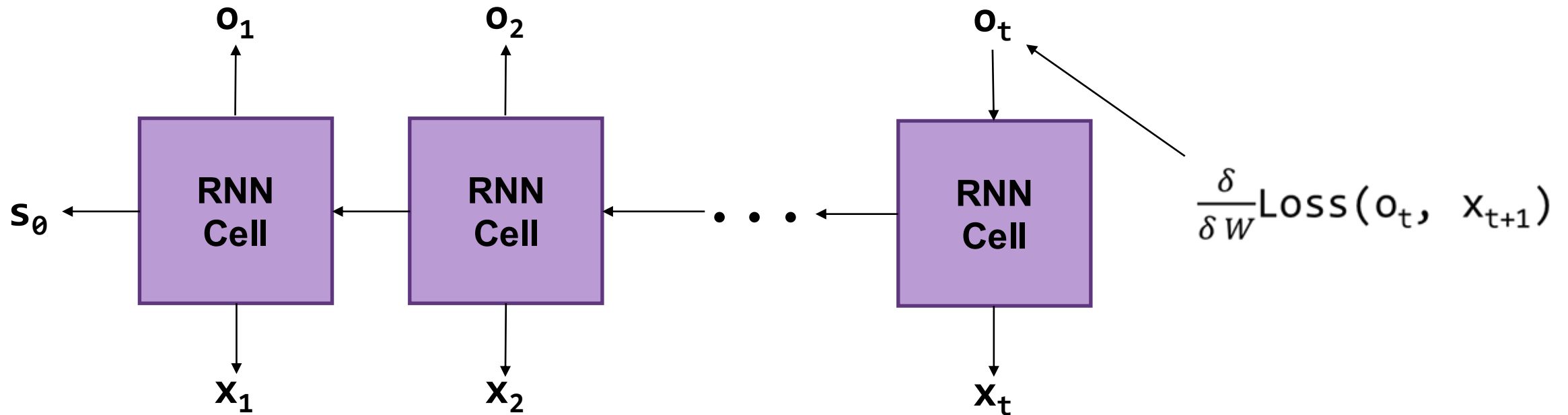
Training RNNs

We can calculate the cross entropy loss just as before since for any sequence of input words ($x_1, x_2, \dots, x_t$), we know the true next word x_{t+1}



Training RNNs

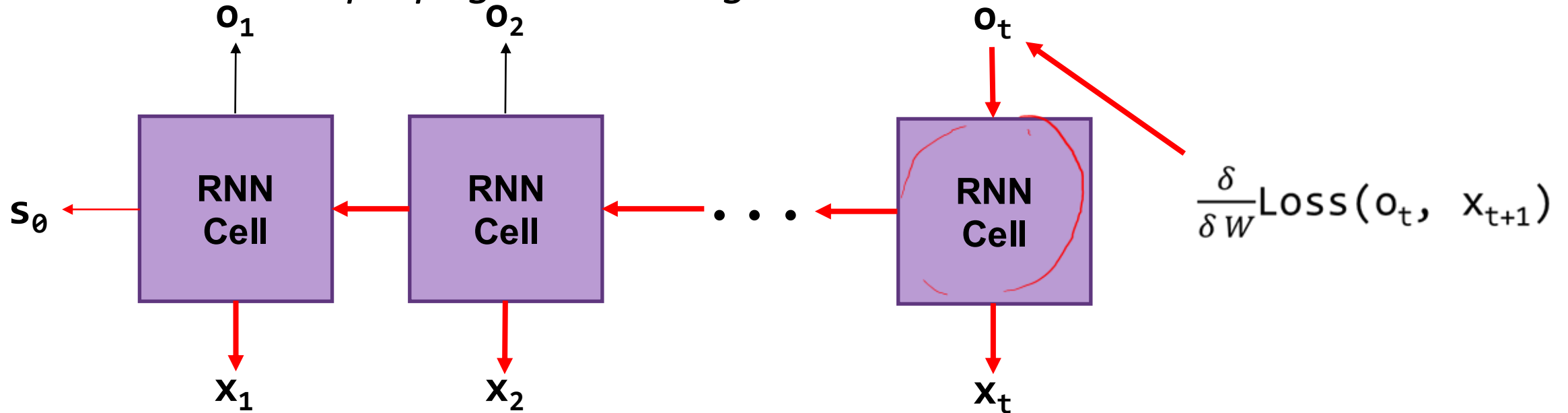
But what happens when we differentiate the loss and backpropagate?



Training RNNs

Not only do our gradients for o_t depend on x_t , but also on all of the previous inputs.

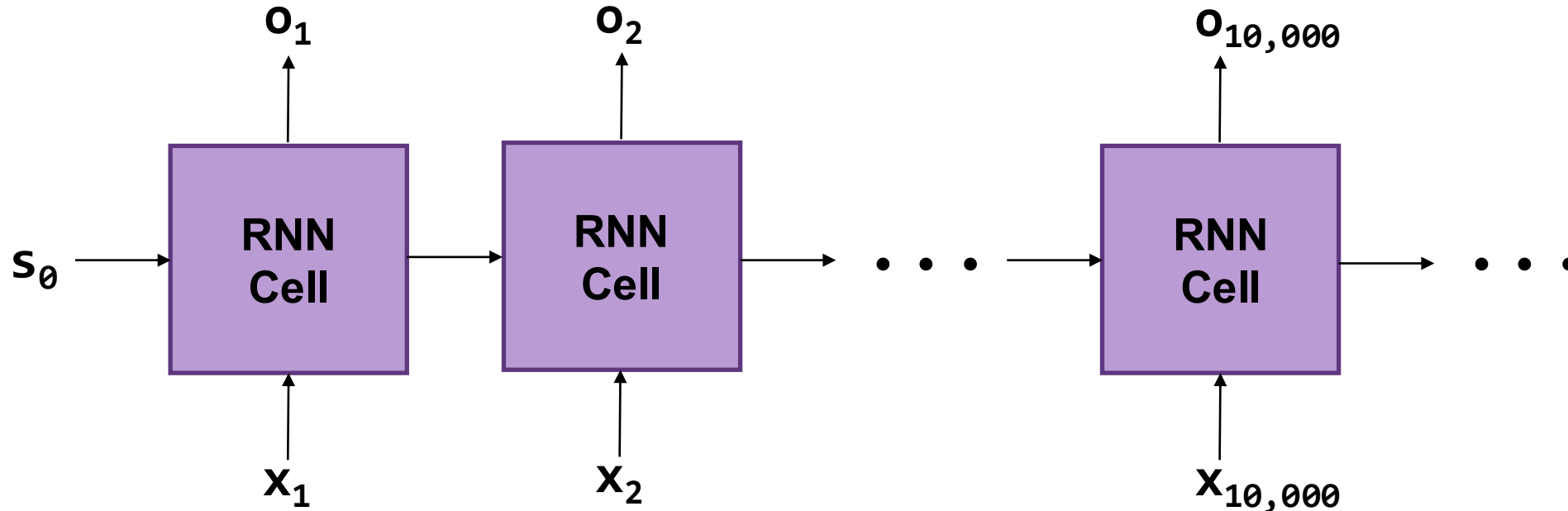
We call this *backpropagation through time*.



Training RNNs

But at what point do we stop and calculate the loss/update?

With this architecture, we can run the RNN cell for as many steps as we want, constantly accumulating memory in the state vector.



Training RNNs

Solution: We define a new hyperparameter called `window_sz`.

We now chop our corpus into sequences of words of size `window_sz`

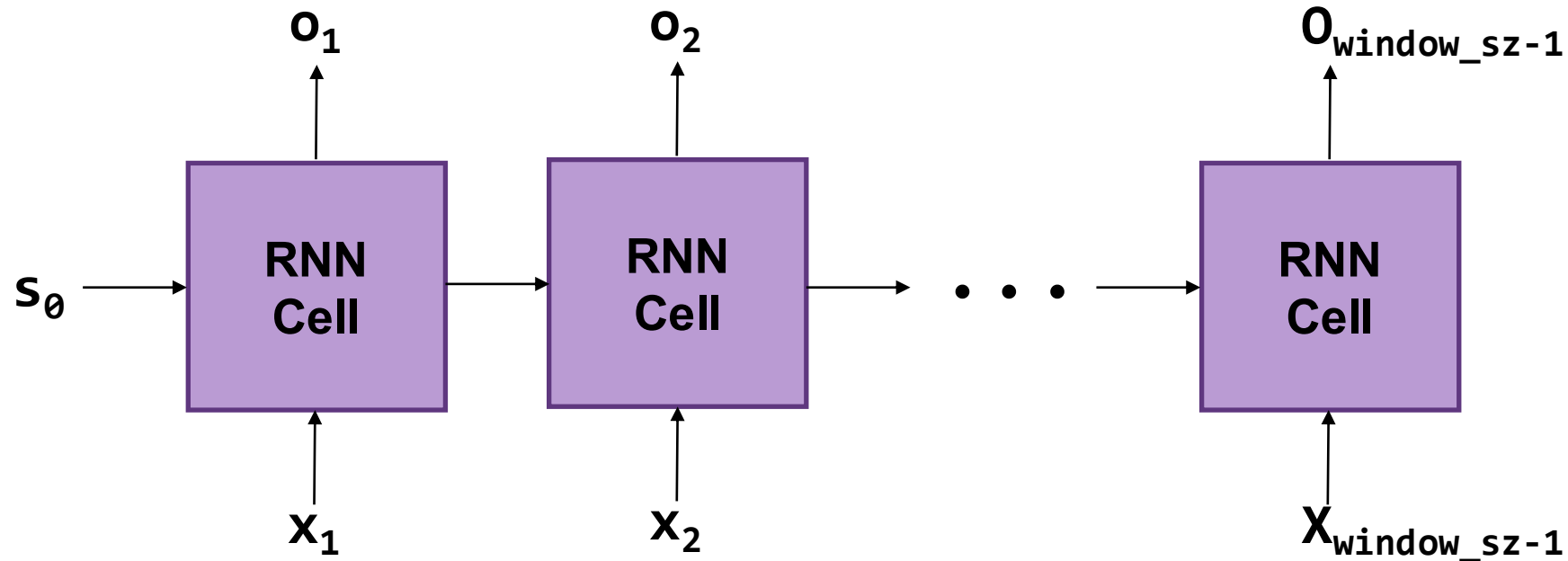
The new shape of our data should be:

`(batch_sz, window_sz, embedding_sz)`

Each example in our batch is a “window” of `window_sz` many words. Since each word is represented as an `embedding_sz`, that is the last dimension of the data.

Training RNNs

Now that every example is a window of words, we can run the RNN till the end of that window, and compute the loss for that specific window and update our weights

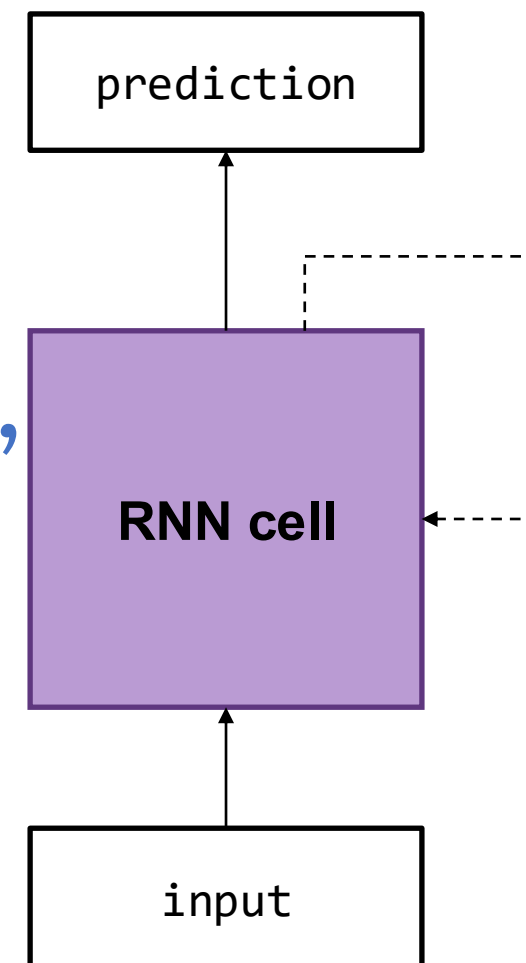
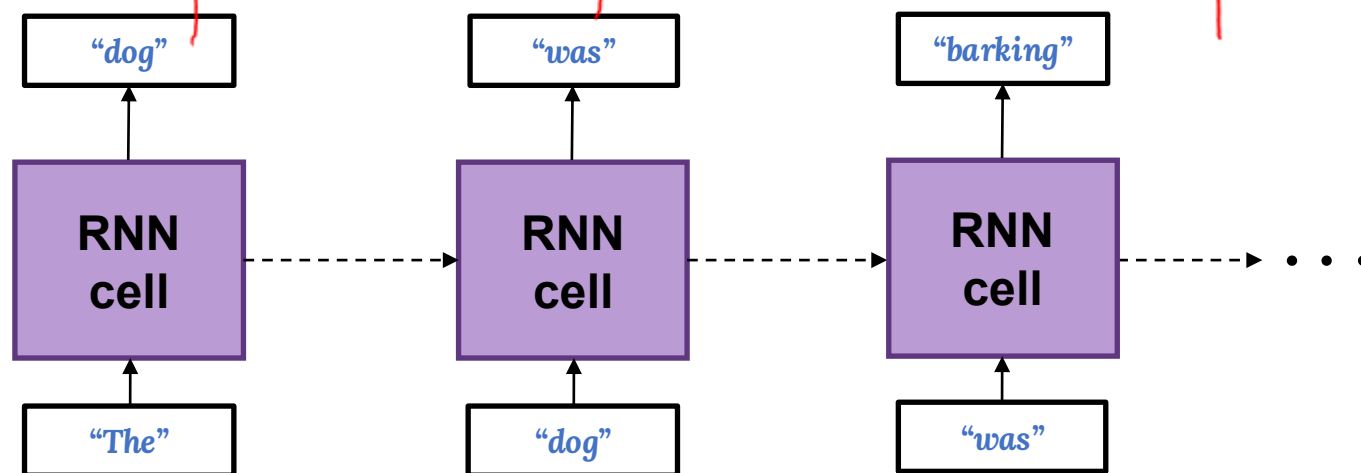




Does RNN fix the limitations of the N-gram model?

1. Number of weights not dependent on N
2. State gives flexibility to choose context from near or far

“The dog was barking at one of the cats.”



RNNs in Pytorch

RNNs can be built from scratch using Python for loops:

```
prev_state = Zero vector
for i from 0 to window_sz:
    state_and_input = concat(inputs[i], prev_state)
    current_state = fc_state(state_and_input)
    outputs[i] = fc_output(current_state)
    prev_state = current_state
return outputs
```


RNNs in Pytorch

RNNs can be built from scratch using Python for loops.

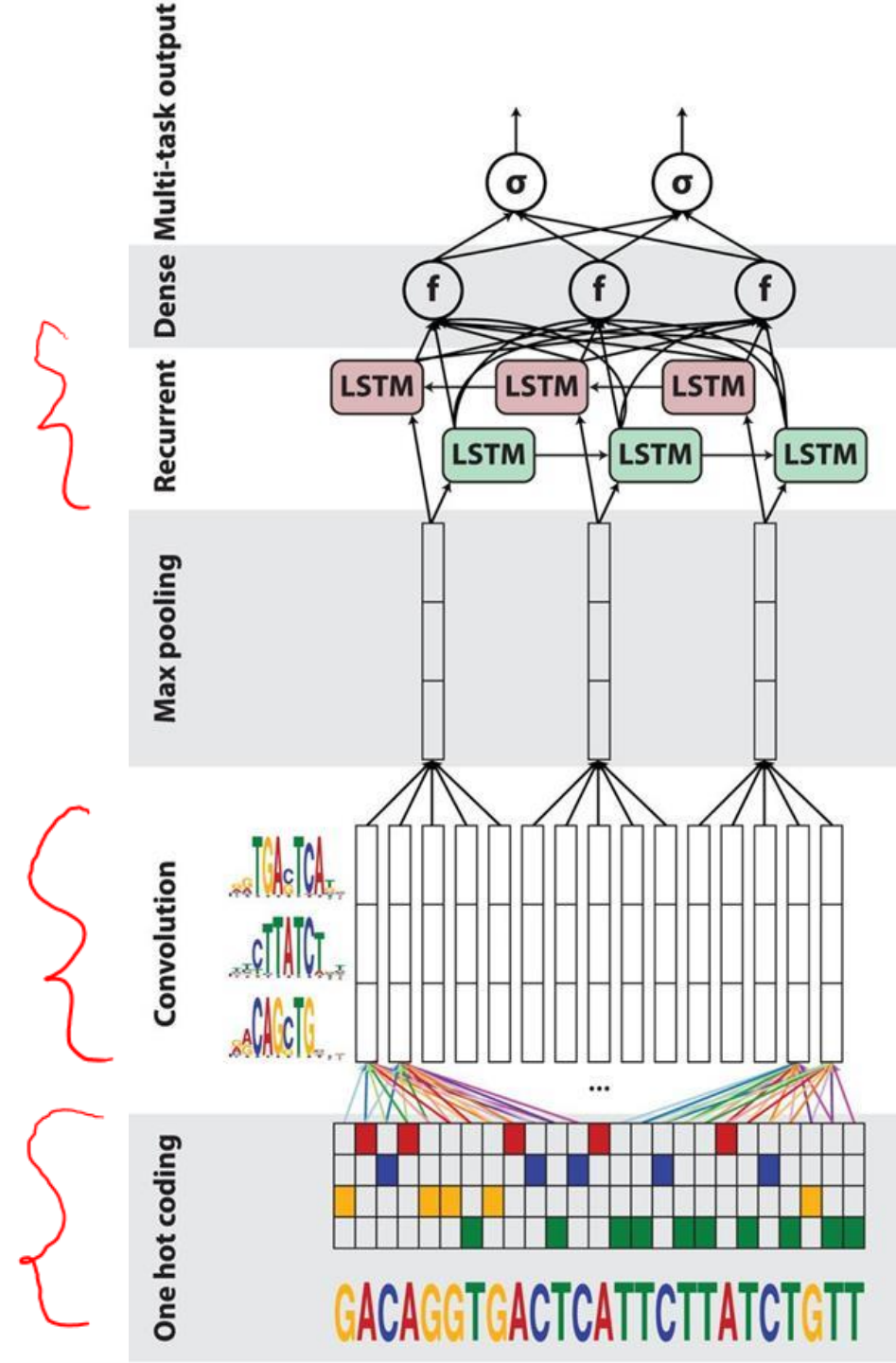
There's also a handy recurrent layer function:

```
class torch.nn.RNN(input_size, hidden_size, num_layers=1, ...nonlinearity='tanh', bias=True, ...,  
                    dropout=0.0, bidirectional=False)
```

embedding

↓

DanQ: a hybrid convolutional and recurrent deep neural network for quantifying the function of DNA sequences



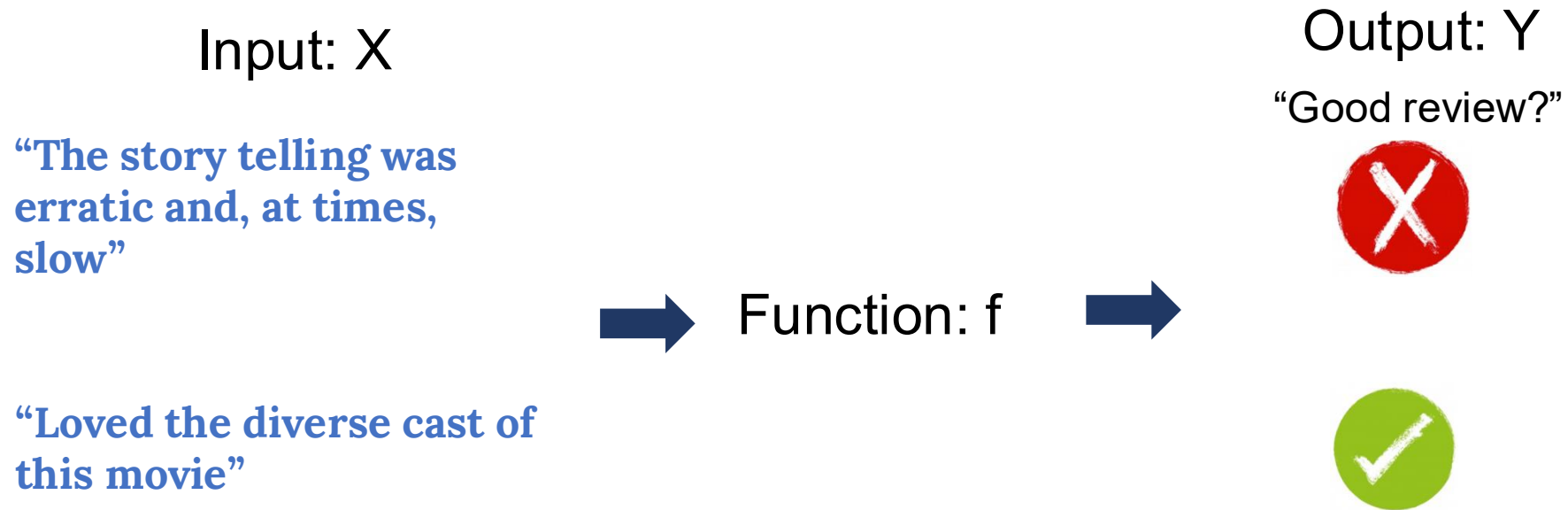
Seq-to-seq modeling

Review: Natural Language Prediction Tasks

*“They went to the grocery store and bought... bread?
milk?
rock?”*

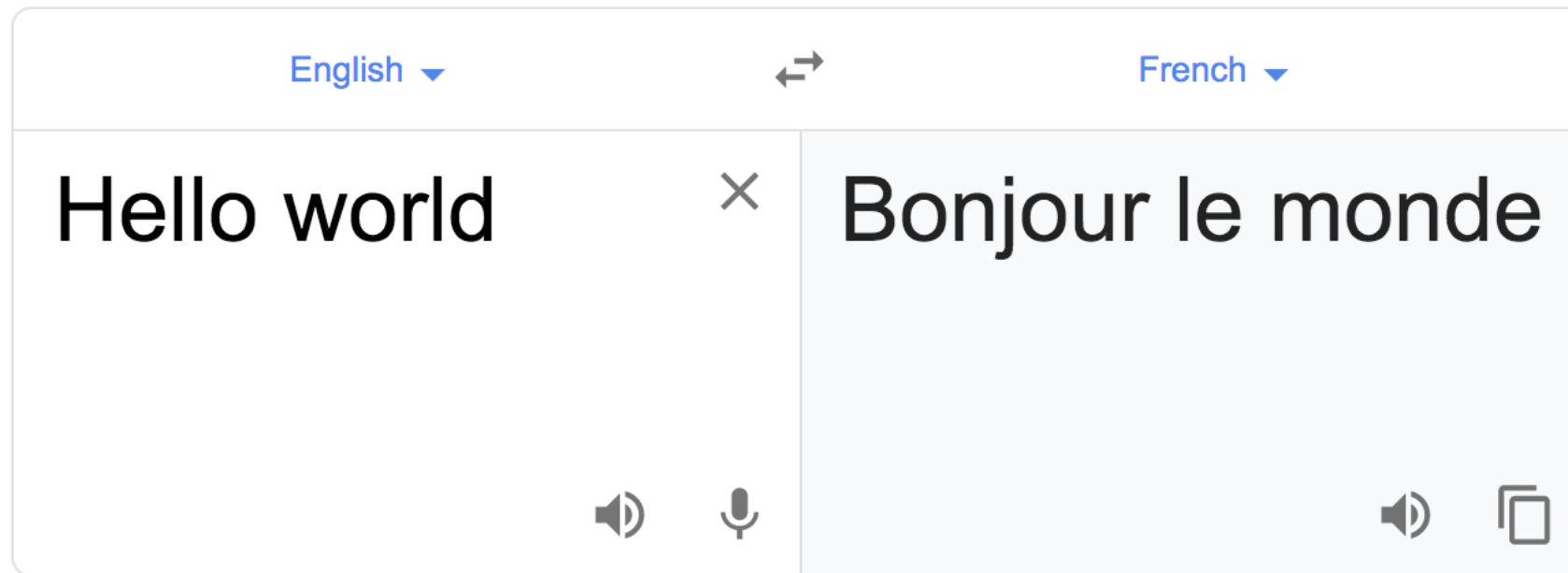
Generating artificial sentences: Here each word is a discrete unit; predicting the next part of the sequence means predicting words

Review: Natural Language Prediction Tasks



Machine Translation (MT)

Software that transforms text in a source language into text in a target language



[Open in Google Translate](#)

[Feedback](#)

Why is this an interesting problem to solve?

- **Complex:** languages evolve rapidly and don't have a clear and well-defined structure
 - Example of language change: “awful” originally meant “full of awe”, but is now strictly negative
- **Important:** billions per year spent on translation services
 - >CA\$2.4 billion spent per year by Canadian government
 - >£100 million spent per year by UK government

Canadian Hansards

- Hansards are transcripts of parliamentary debates
- Canada's official languages are English and French, so everything said in parliament is transcribed in both languages



Canadian Hansards: Examples

English	French
What a past to celebrate.	Nous avons un beau passé à célébrer.
We are about to embark on a new era in health research in this country.	Le Canada est sur le point d'entrer dans une nouvelle ère en matière de recherche sur la santé.

Example from Hansards

- For example, take the first entry in Hansard's:

edited hansard number 1

hansard révisé numéro 1

LM approach

- Language modelling works on a word-by-word basis, taking only previous words as input

$$P(w_{t,i}) = P(w_{t,i} \mid \underbrace{w_{s,i-1}, w_{s,i-2}, \dots, w_{s,0}})$$


- Where $w_{t,i}$ is the i^{th} word in the target sentence, and $w_{s,i}$ is the i^{th} word in the source sentence

Will it work for MT task?

Why our LM approach doesn't work for MT

- Language modelling works on a word-by-word basis, taking only previous words as input

$$P(w_{t,i}) = P(w_{t,i} \mid w_{s,i-1}, w_{s,i-2}, \dots, w_{s,0})$$

- Where $w_{t,i}$ is the i^{th} word in the target sentence, and $w_{s,i}$ is the i^{th} word in the source sentence
- However, **it is not a given that the information we need comes in the preceding words**
- The order and length of the source and target sentences are not necessarily equal

Example from Hansards

- For example, take the first entry in Hansard's:

edited hansard number 1



hansard révisé numéro 1

Further examples

French: “Londres me manque”

Naive translation: “London I miss”

Correct translation: “I miss London”

French: “Je viens de partir”

Naive translation: “I come of to go”

Correct translation: “I just left”

Sequence to Sequence (seq2seq)

Thus, we cannot simply use the previous words – we need to ***summarize the source sentence first***

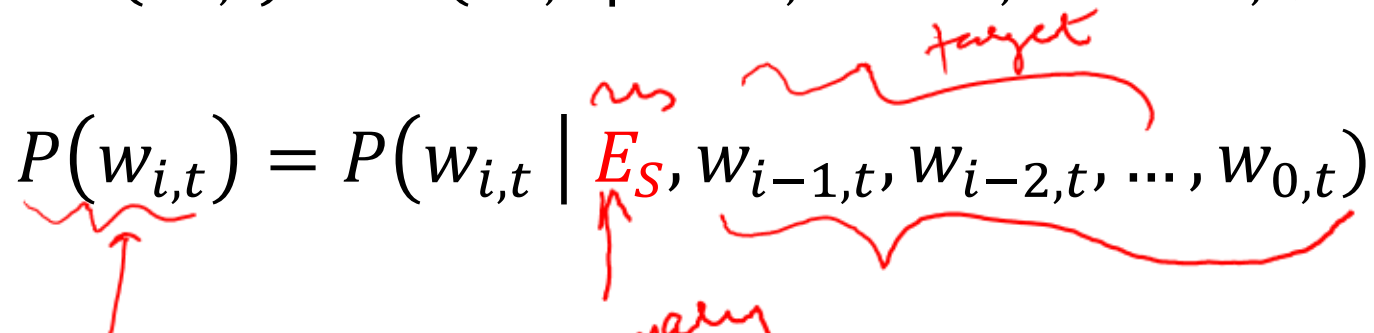
This is called sequence to ***sequence to sequence learning***, or ***seq2seq***

Sequence to Sequence (seq2seq)

Instead of:

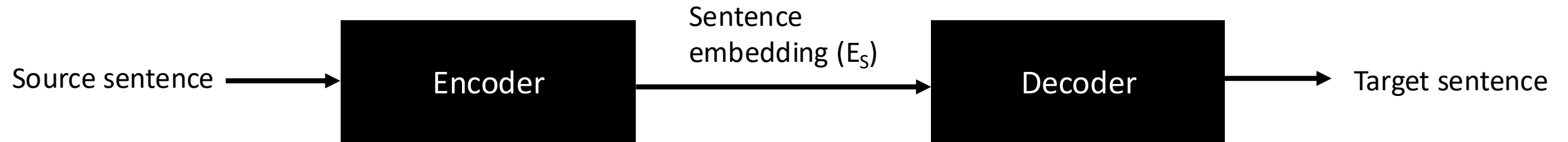
$$P(w_{i,t}) = P(w_{i,t} \mid w_{i-1,s}, w_{i-2,s}, \dots, w_{0,s})$$

Let's do:


$$P(w_{i,t}) = P(w_{i,t} \mid E_s, w_{i-1,t}, w_{i-2,t}, \dots, w_{0,t})$$

Where E_s is a summary, or **embedding**, of the sentence taken from the source language, and w_i is the i^{th} word of the sentence in the target language

What will the neural net look like?

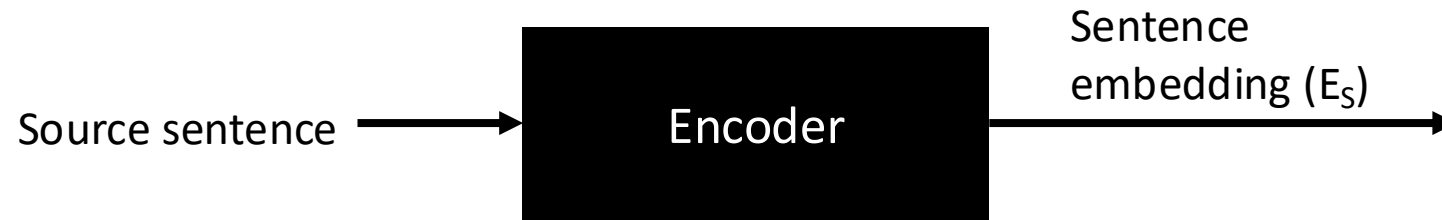


Origin of the encoder/decoder terminology: information theory

- The encoder “compresses” the source sentence into a compact “code”
- The decoder recovers the sentence (but in the target language) from this code

What will the neural net look like?

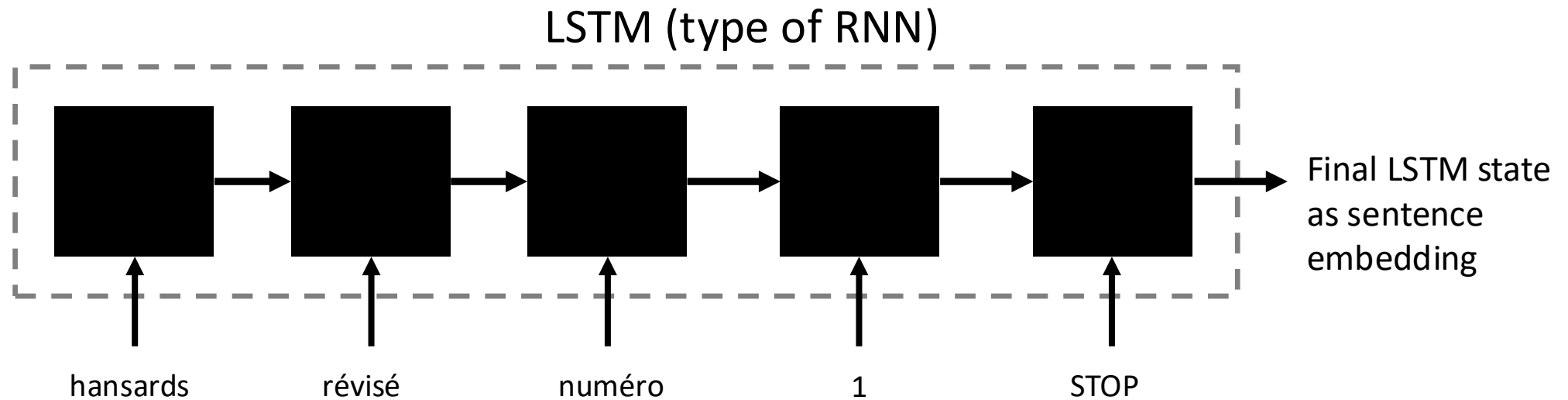
Any ideas?



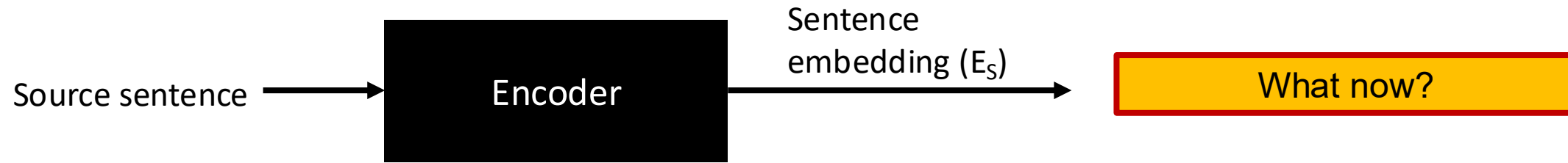
Encoder

- To generate the sentence embedding, we need an encoder
- Use an RNN
- Feed in the source sentence
- Take the final RNN state as the sentence embedding
- This will be a ***language-agnostic*** representation of the sentence
 - i.e. it will represent the *meaning* of the sentence without being tied to any particular language

Encoder architecture

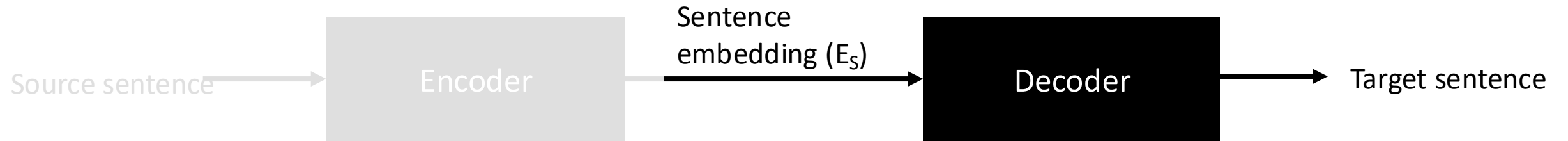


What will the neural net look like?



What will the neural net look like?

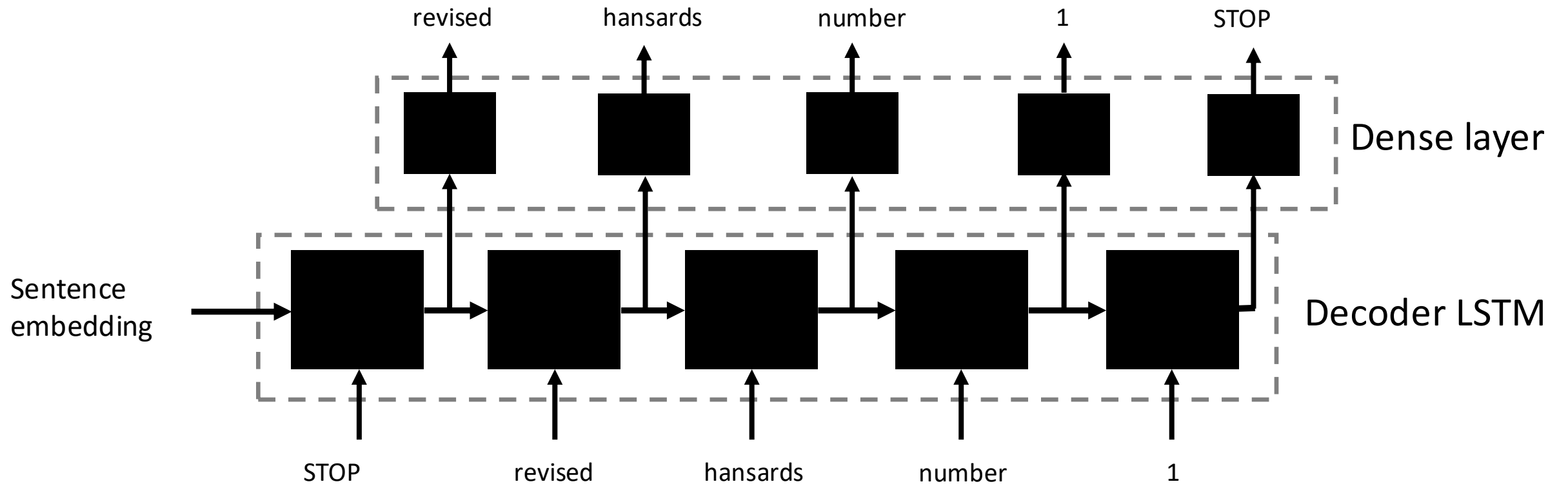
Any ideas?



Decoder

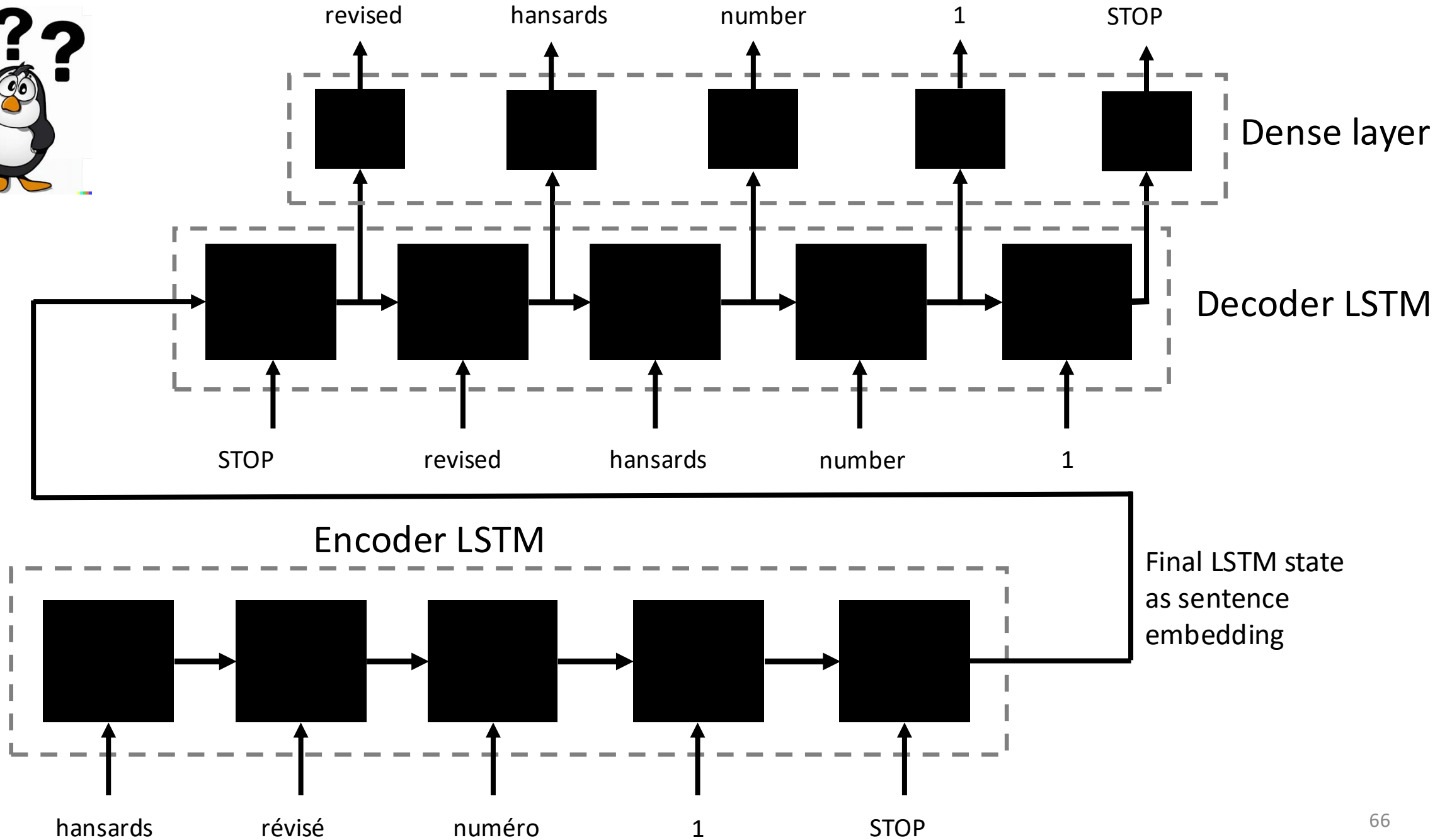
- We now have a sentence embedding representing the meaning of the source sentence
- Now, let's generate a sentence in the target language with the same meaning
- Use an RNN again, **with the sentence embedding** as its initial hidden state
- The rest is just like language modeling:
 - Input to the RNN is the previous word from the target sentence
 - Take each RNN output and put it through a fully connected layer
 - Softmax to convert to probability distribution over next word in target language

Decoder architecture



Putting it all together...

Any questions?



Transformers

“Attention Is All You Need”

A 2017 paper that introduced the **Transformer** model for machine translation

- Has no recurrent networks!
- **Only** uses attention

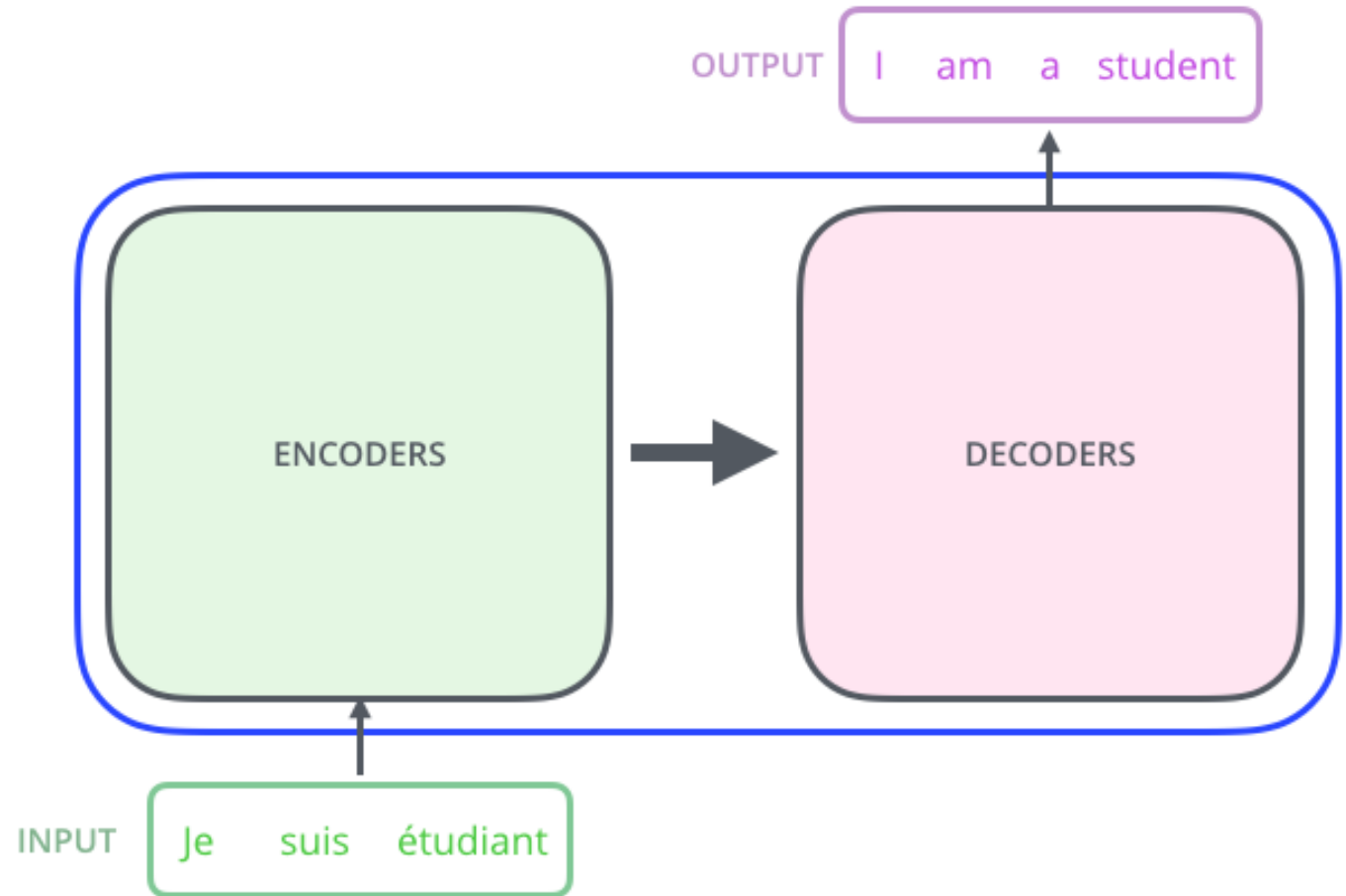


Motivation:

- RNN training is hard to parallelize since the previous word must be processed before next word
 - Transformers are trivially parallelizable
- Preserving important **linguistic context over very long sequences is difficult**
 - Transformers don't even try to remember things (every step looks at a weighted combination of **all** words in the input sentence)

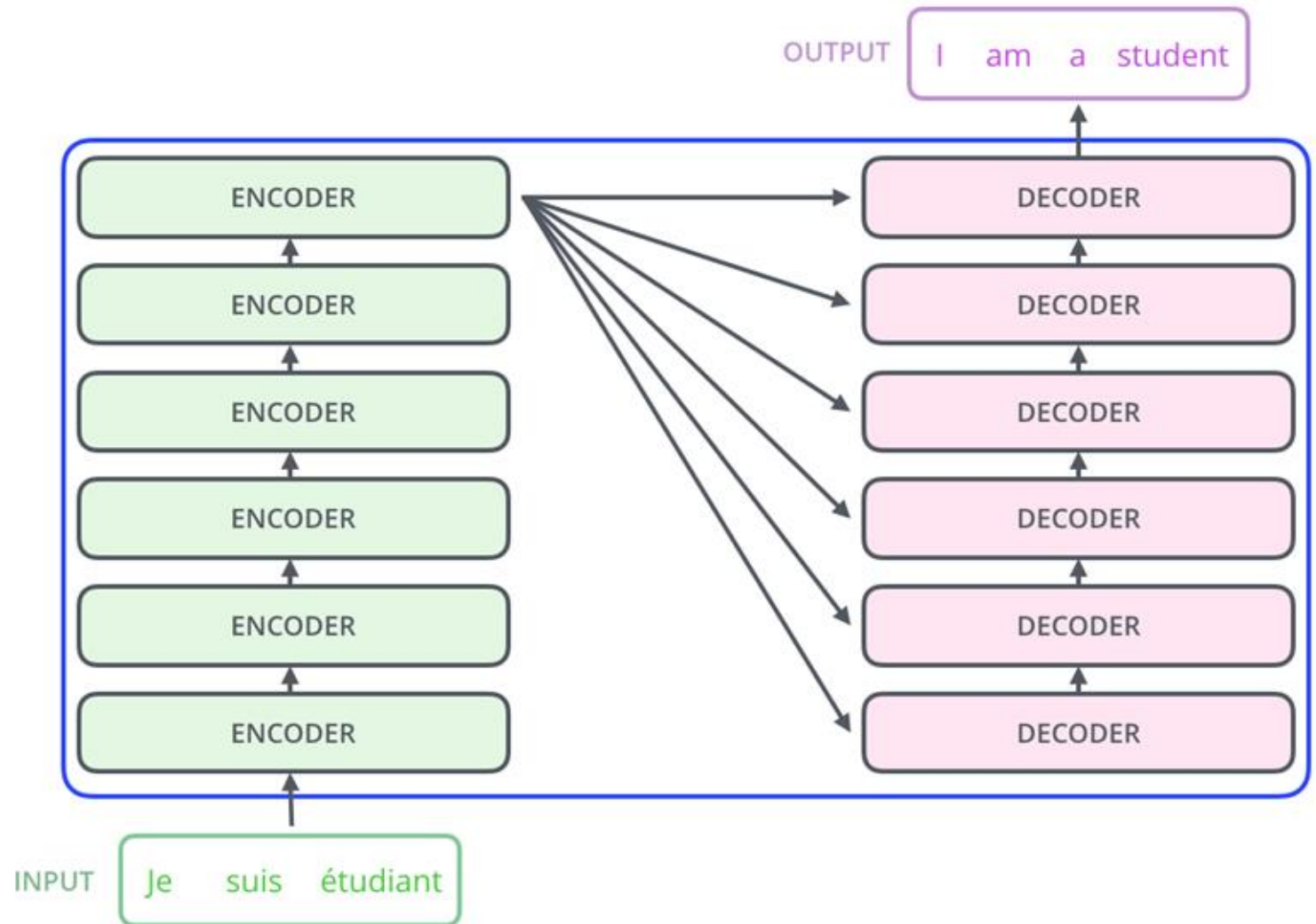
Transformer Model Overview

- The Transformer model breaks down into Encoder and Decoder blocks.
- At a high level, similar to the seq2seq architecture we've seen already...
- ...but there are no recurrent nets inside the Encoder and Decoder blocks!



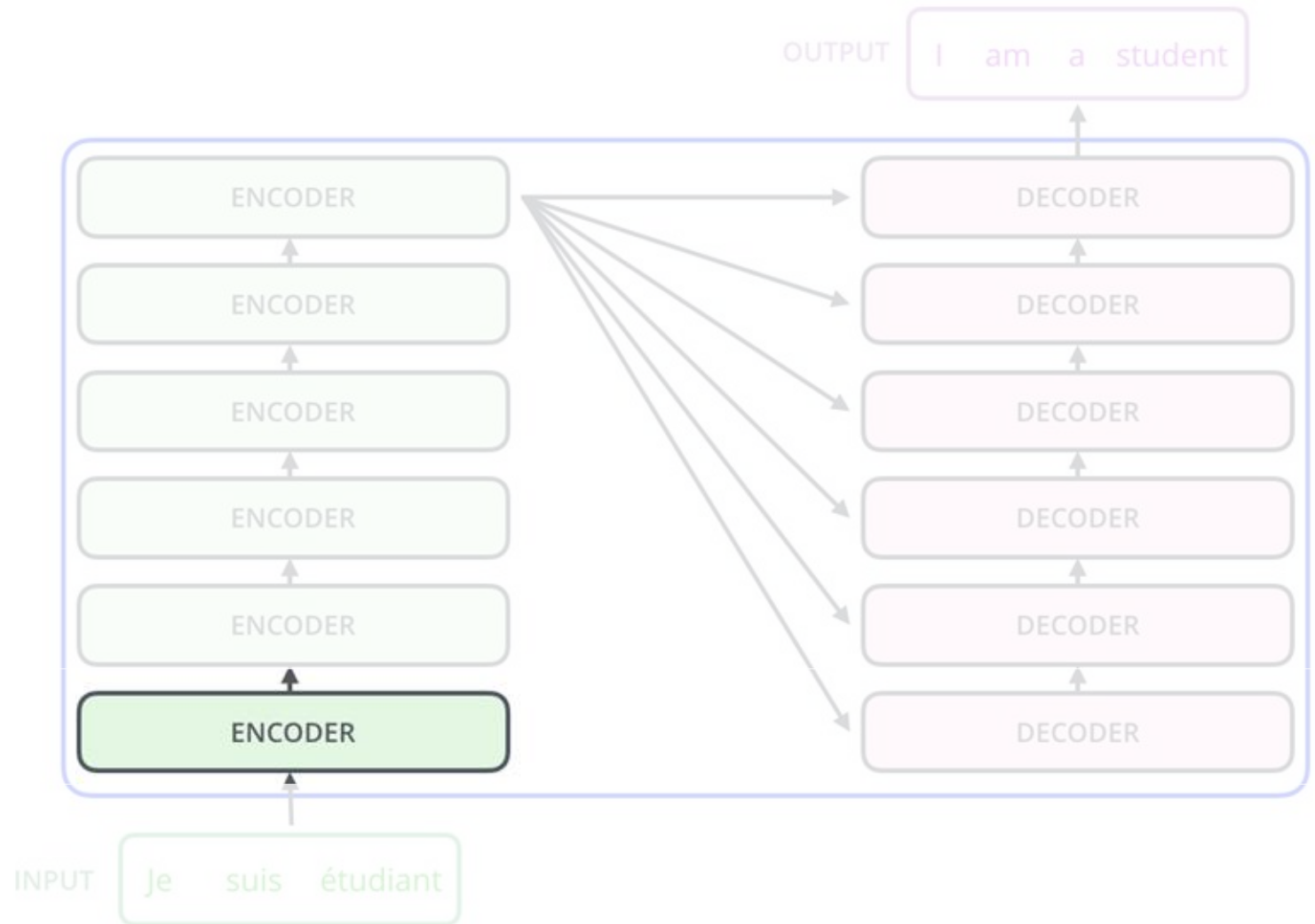
Transformer Model Overview

- The Transformer model breaks down into Encoder and Decoder blocks.
- At a high level, similar to the seq2seq architecture we've seen already...
- ...but there are no recurrent nets inside the Encoder and Decoder blocks!
- For better performance, often stack multiple Encoder and Decoder blocks (deeper network)



Transformer Model Overview

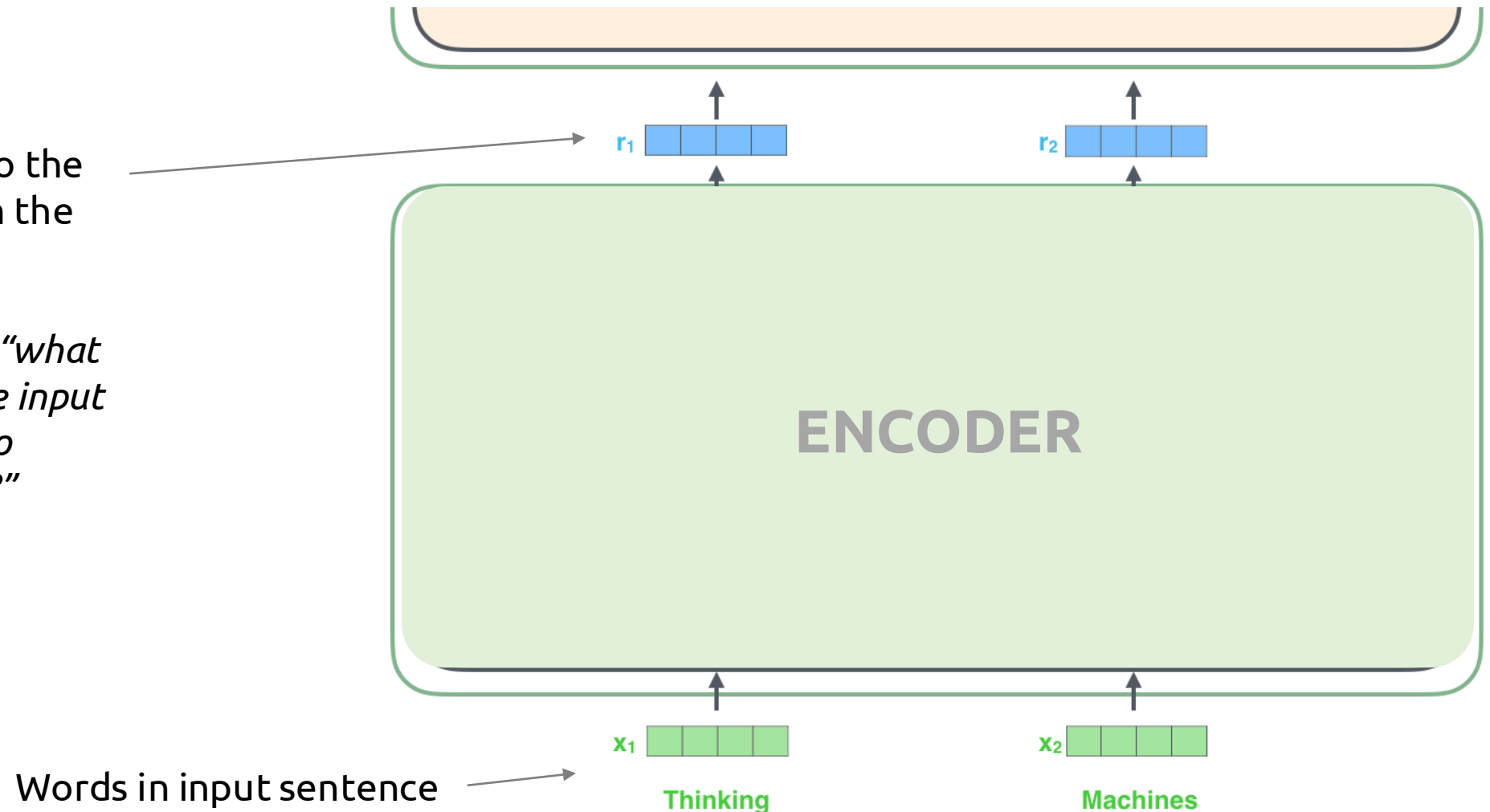
- Let's look at what goes on inside one of these Encoder blocks



Encoder Block Map

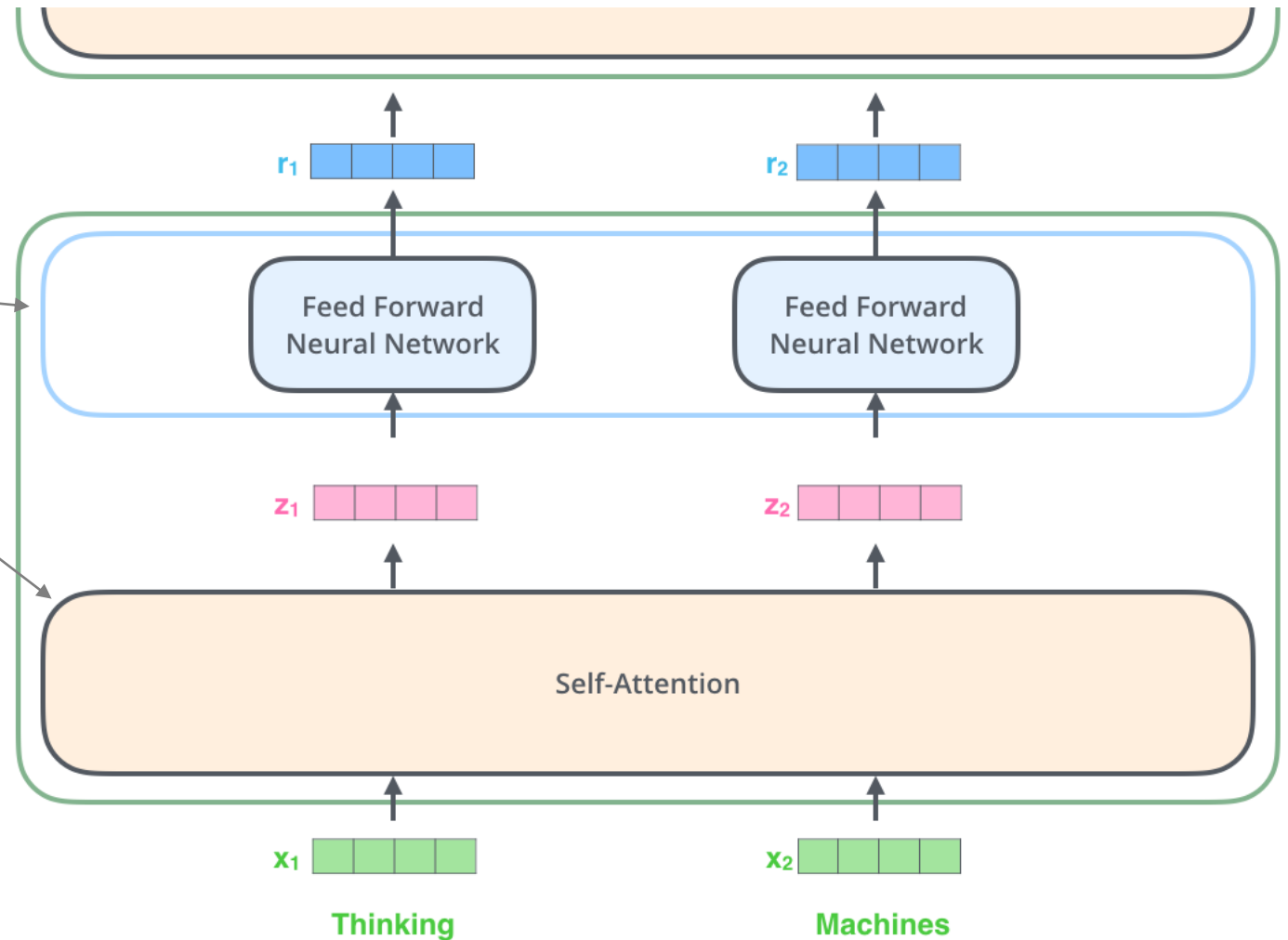
These per-word output vectors are analogous to the RNN hidden states from the seq2seq model

- They should capture “*what information about the input sentence is relevant to translating this word?*”



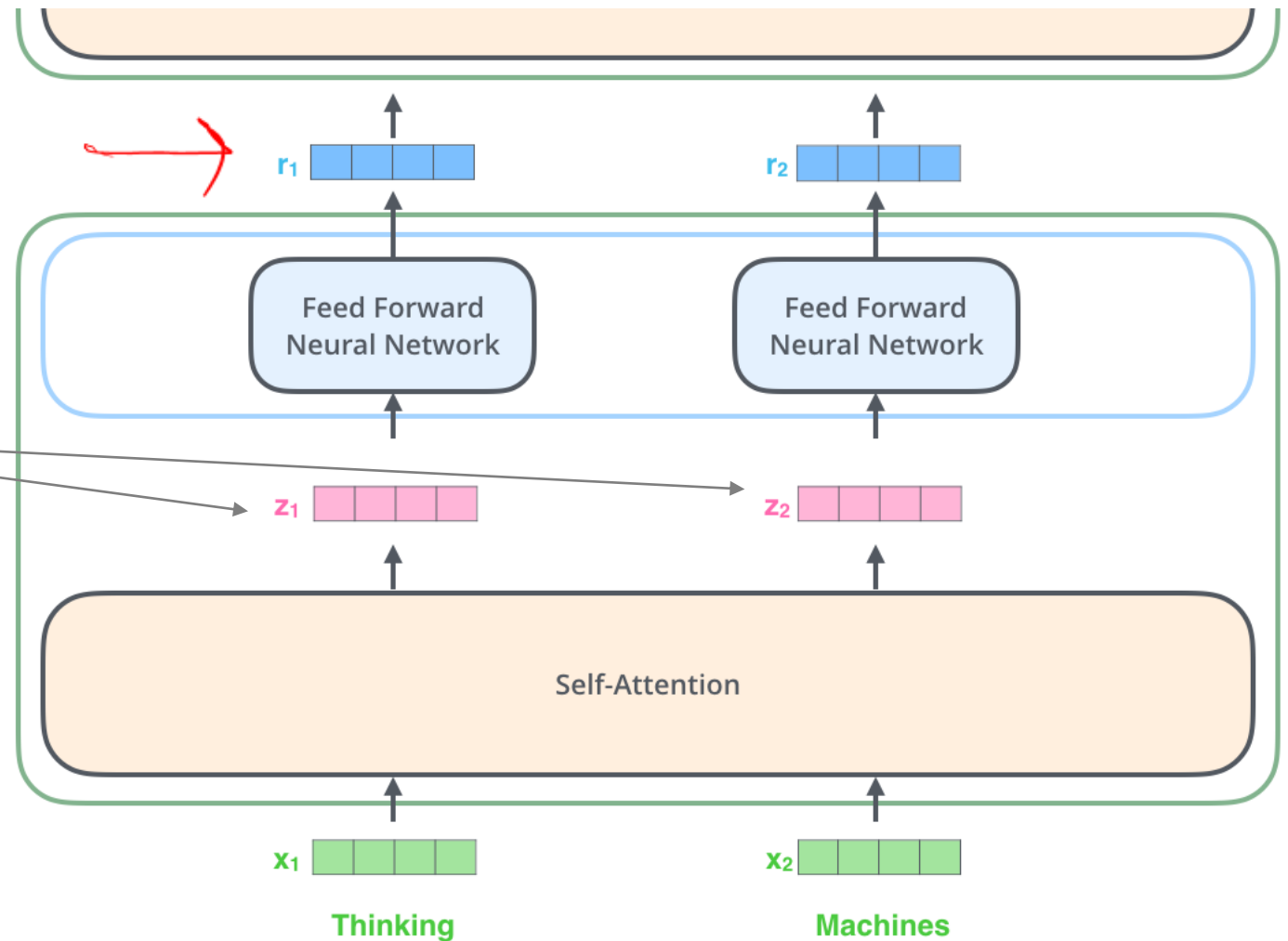
Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.



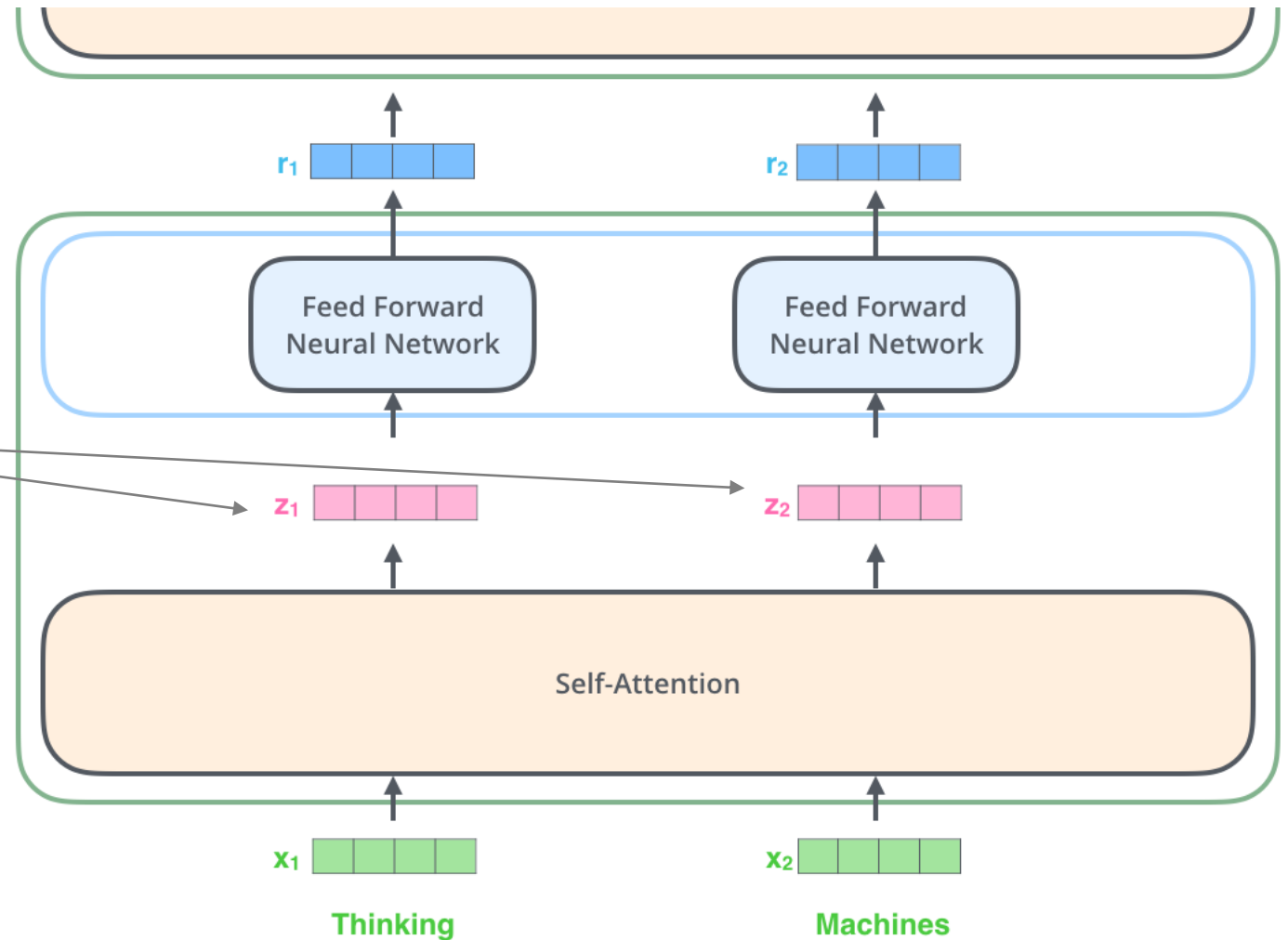
Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.
- Self-Attention layer is applied to each word individually.



Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.
- Self-Attention layer is applied to each word individually.



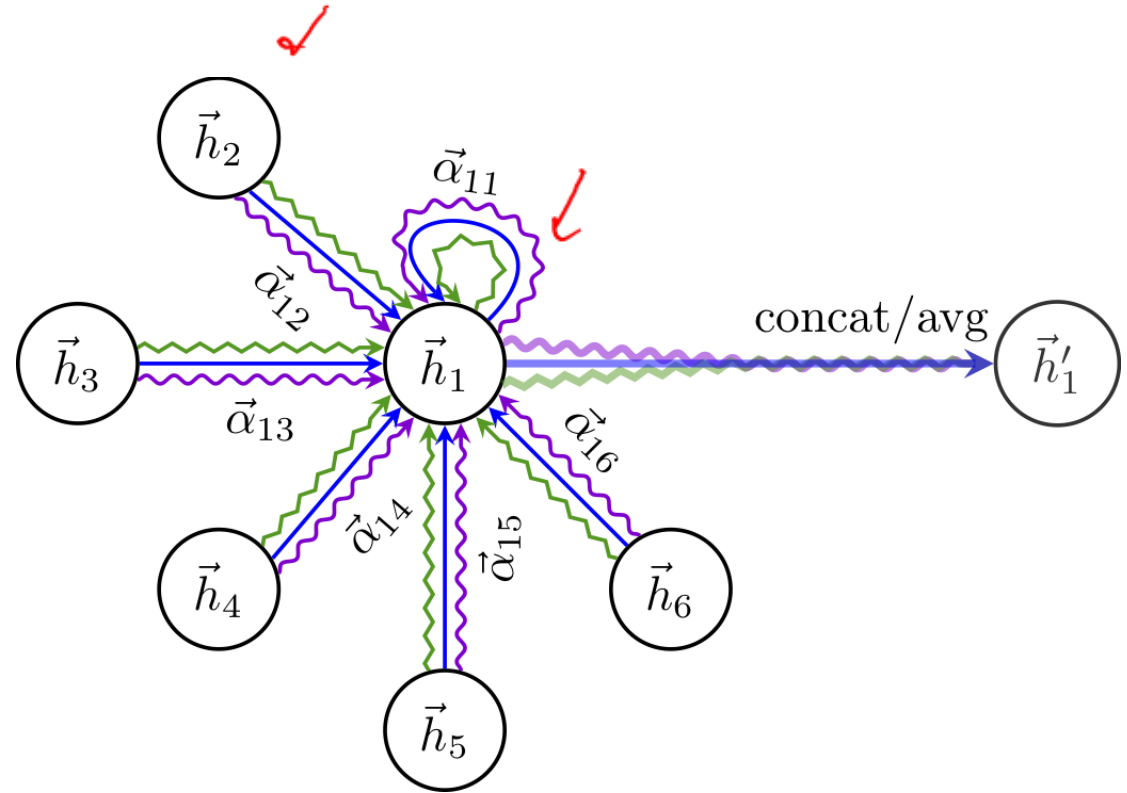
Review: Graph Attention Networks (GAT)

Let's walk through attention steps:

(1) Alignment score $e_{ij} = a(\vec{h}_i, \vec{h}_j)$

(2) Softmax
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}$$

(3) Weighted aggregation
$$\vec{h}'_i = \parallel_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{w}^k \vec{h}_j \right)$$



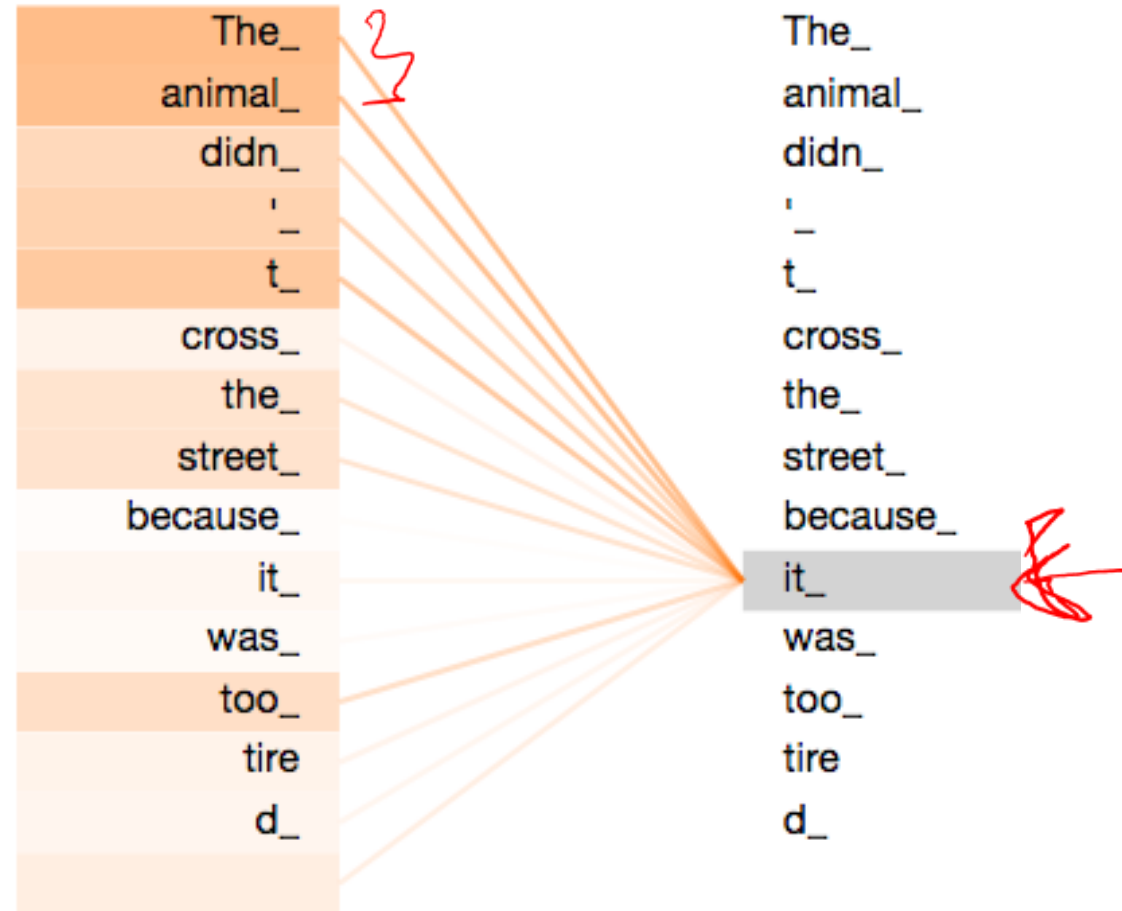
Self-Attention: Input's attention on itself

What do we do next?

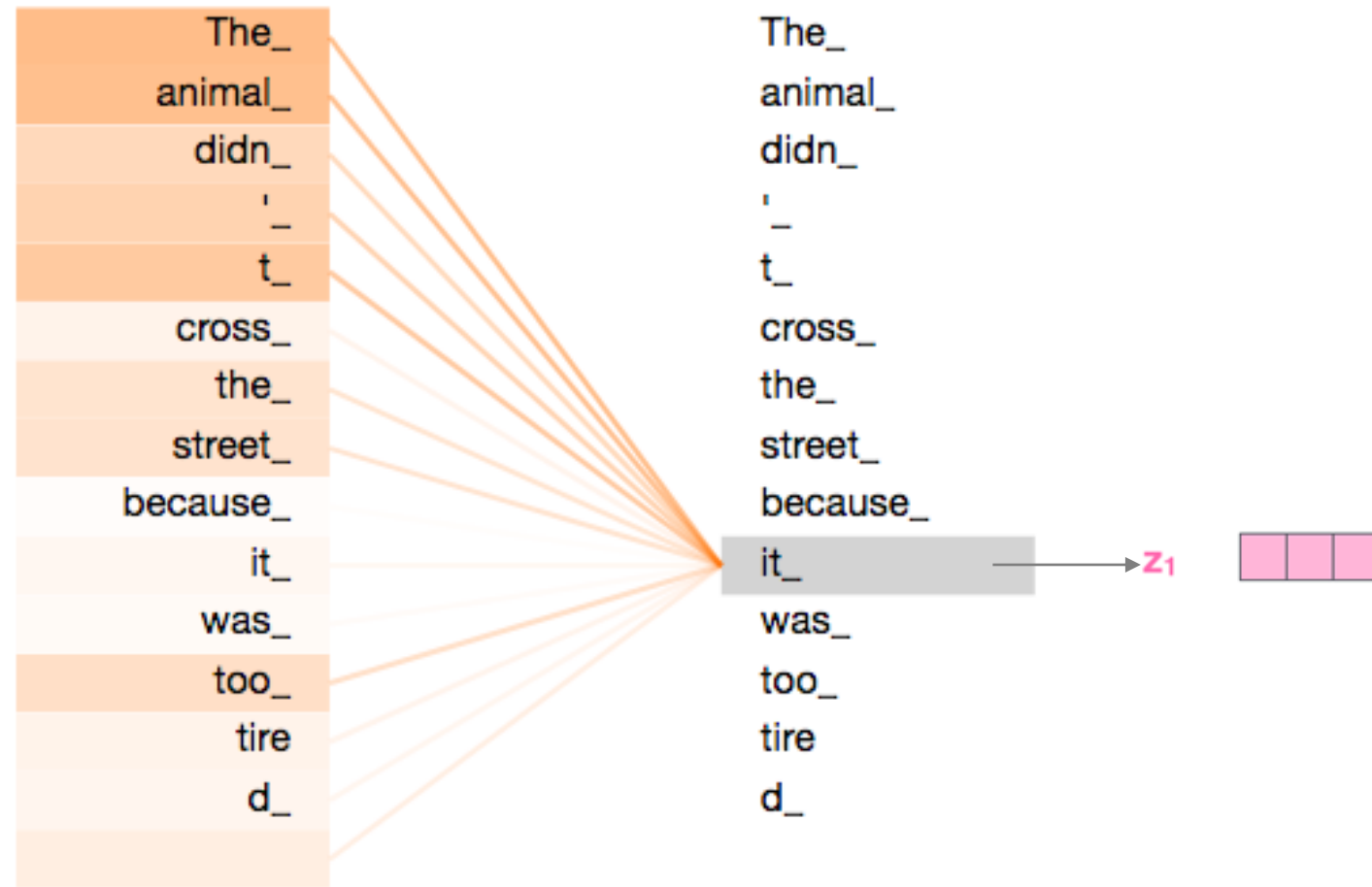
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_

Self-Attention: Input's attention on itself

What do we do next?



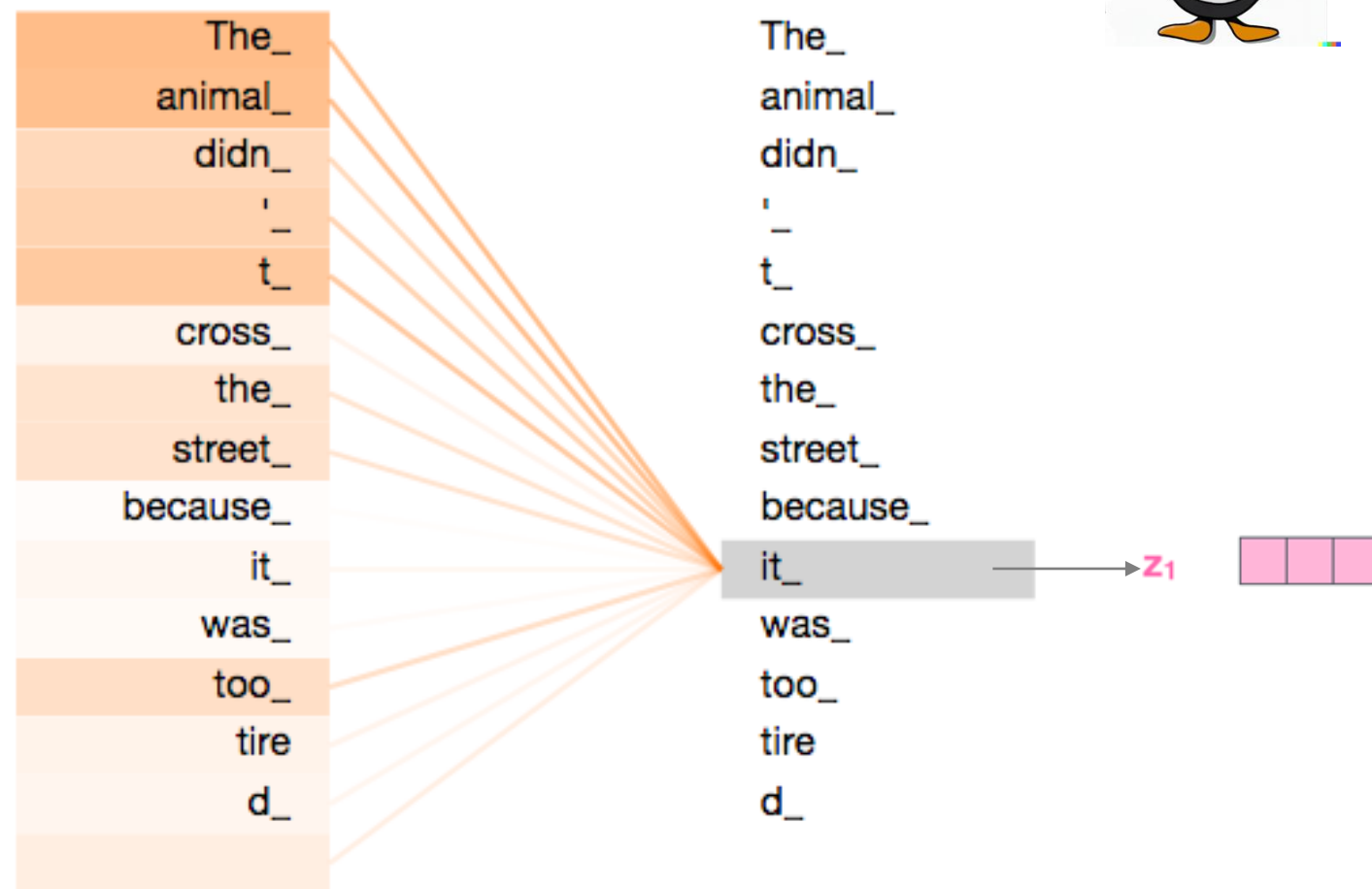
Self-Attention: Input's attention on itself



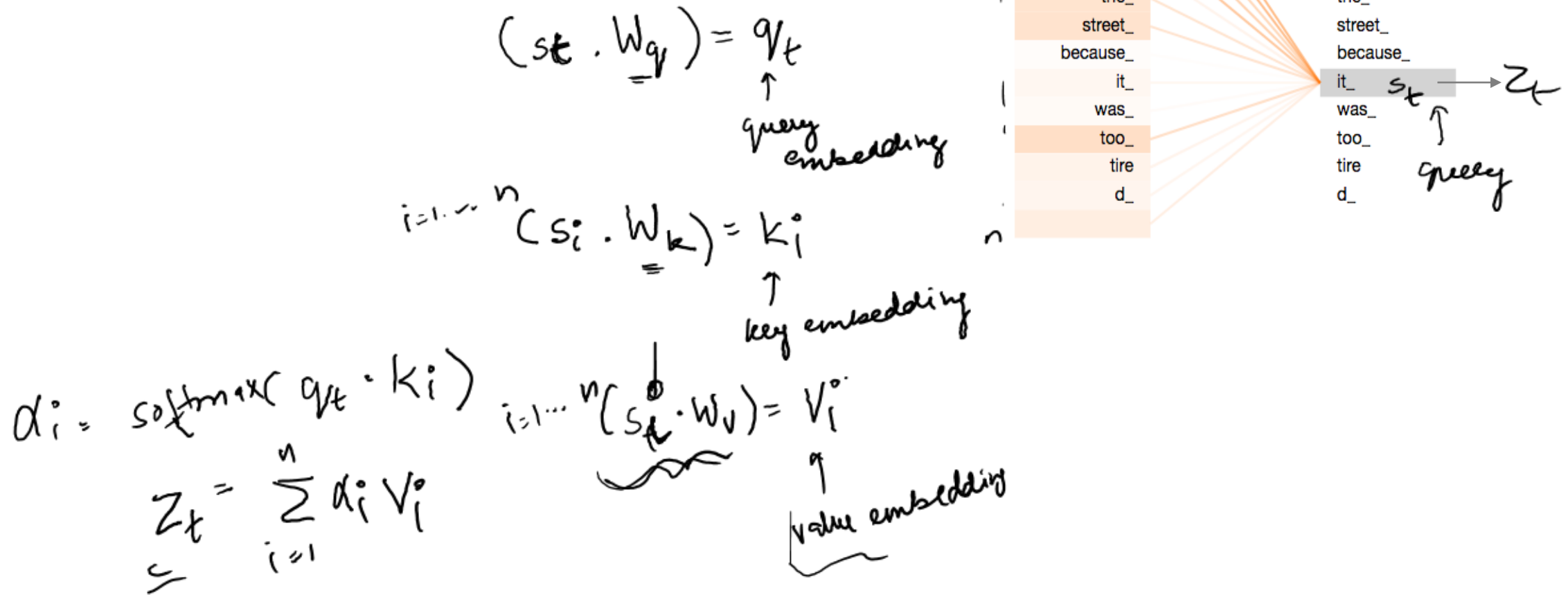


Self-Attention: Overview

- **The big idea:**
Self-attention computes the output vector z_i for each word via a weighted sum of vectors extracted from each word in the input sentence
- Here, self-attention learns that “it” should pay attention to “the animal” (i.e. the entity that “it” refers to)
- Why the name ***self***-attention?
This describes attention that the input sentence pays to itself



Self-Attention: Sketch

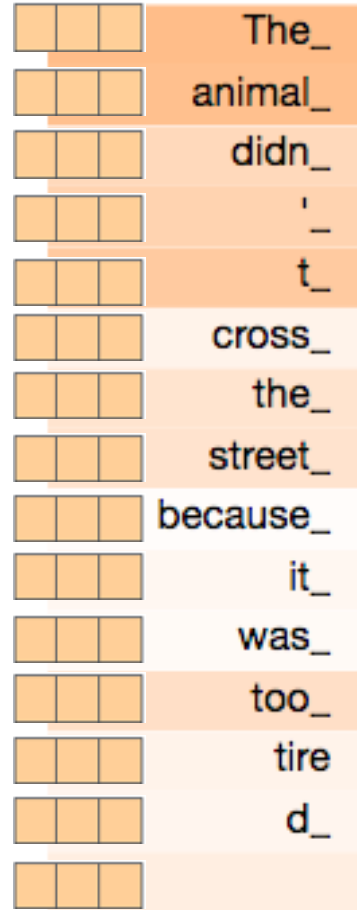


Self-Attention: Overview

How it works:

1. To determine how much attention a word should pay to each other other, we compute a **query vector** for the word and compare it to a **key vector** for every other word...

Key vectors



The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_

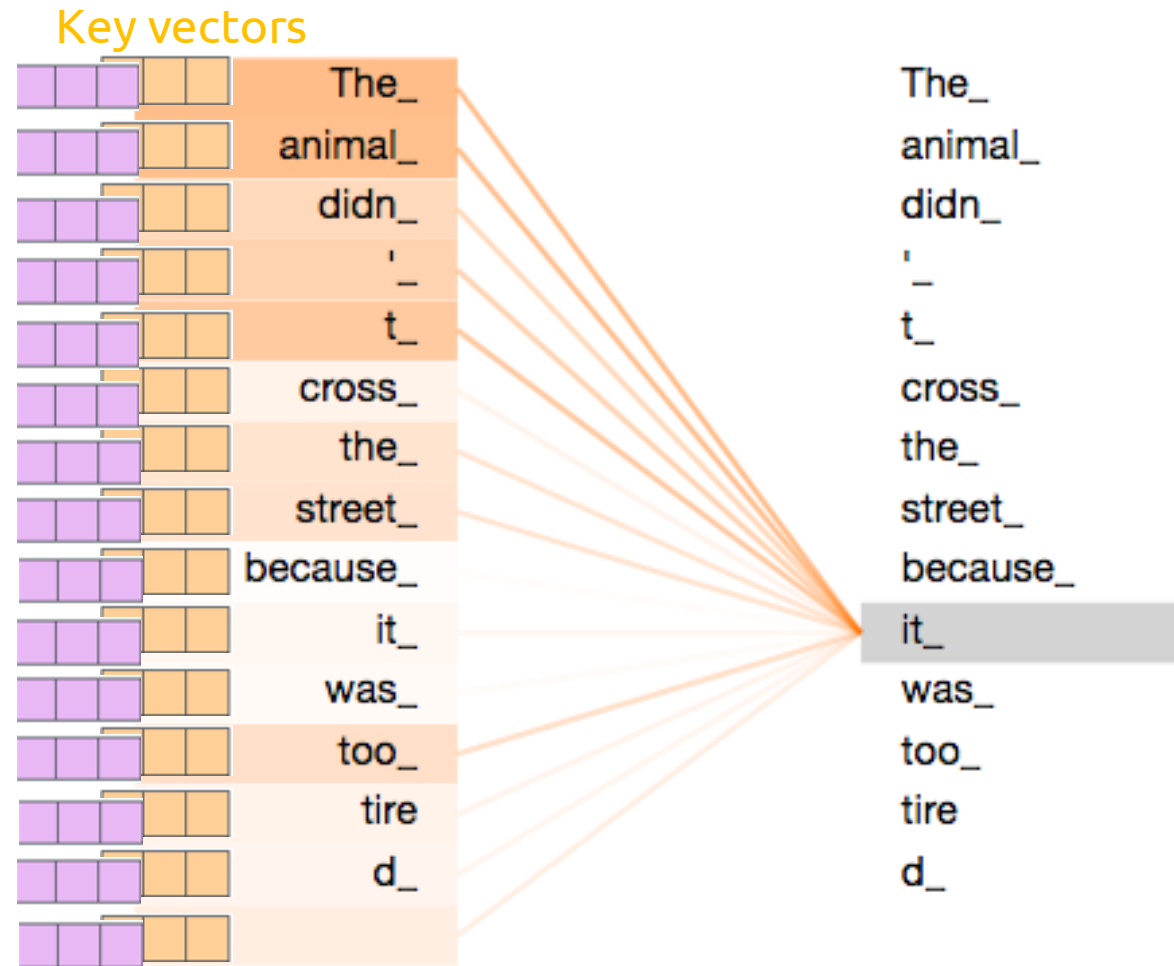
Query vector



Self-Attention: Overview

How it works:

1. To determine how much attention a word should pay to each other other, we compute a **query vector** for the word and compare it to a **key vector** for every other word...

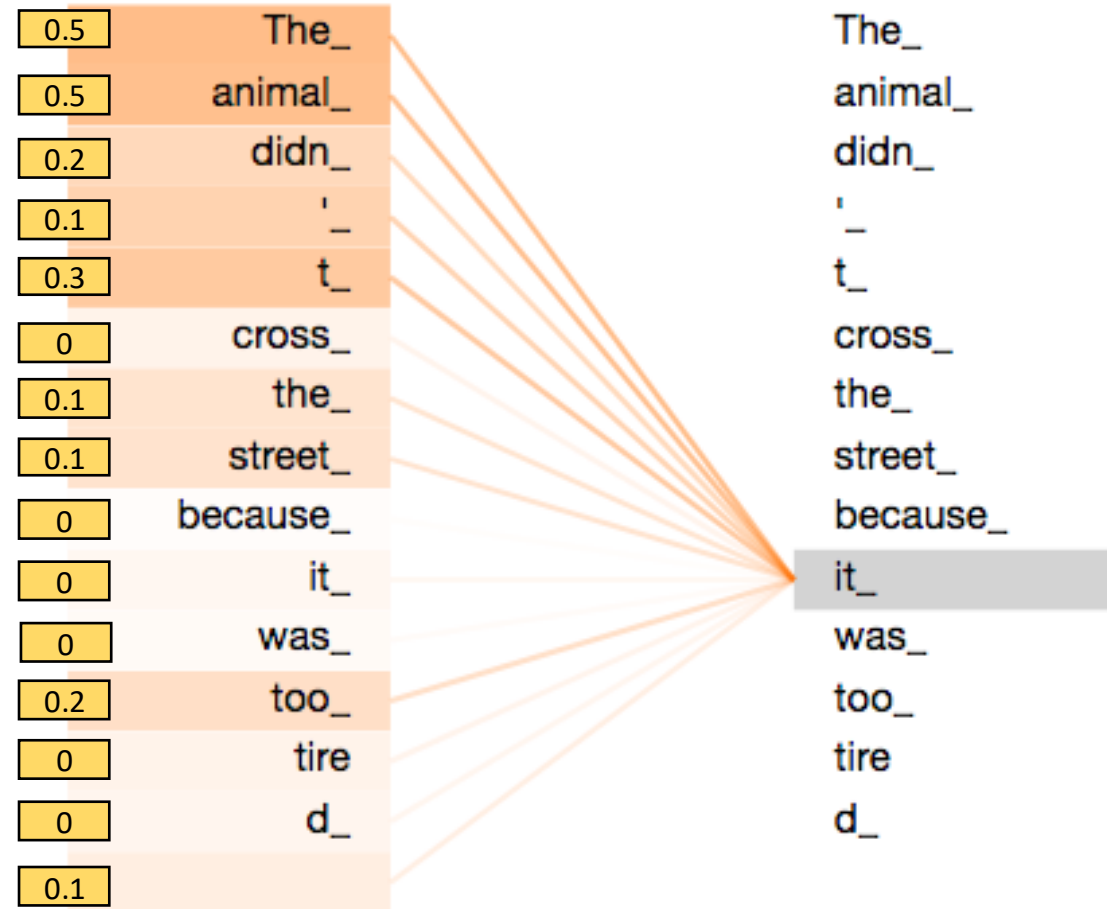


Self-Attention: Overview

How it works:

1. To determine how much attention a word should pay to each other other, we compute a **query vector** for the word and compare it to a **key vector** for every other word... **to compute our alignment score**

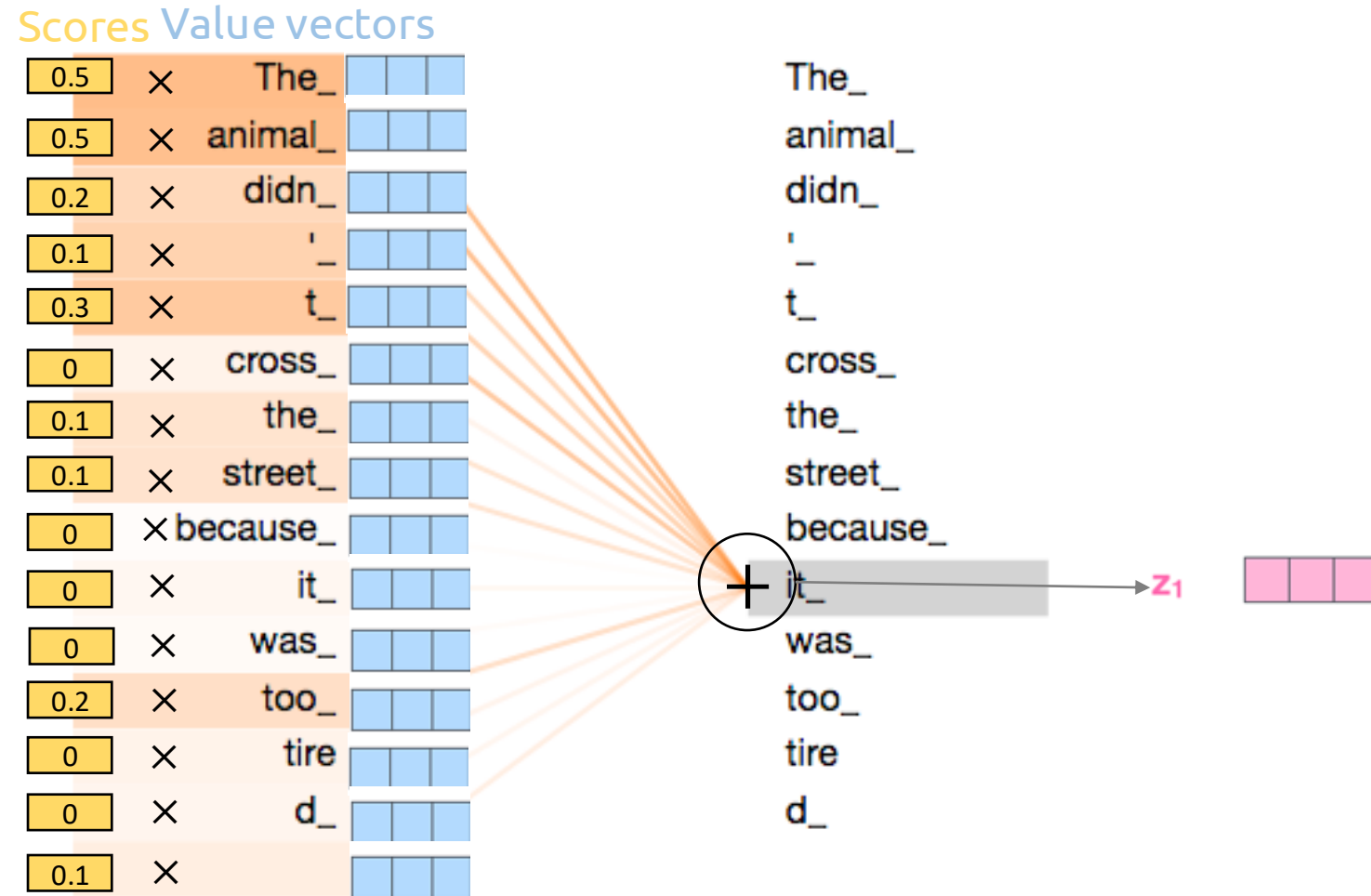
Scores



Self-Attention: Overview

How it works:

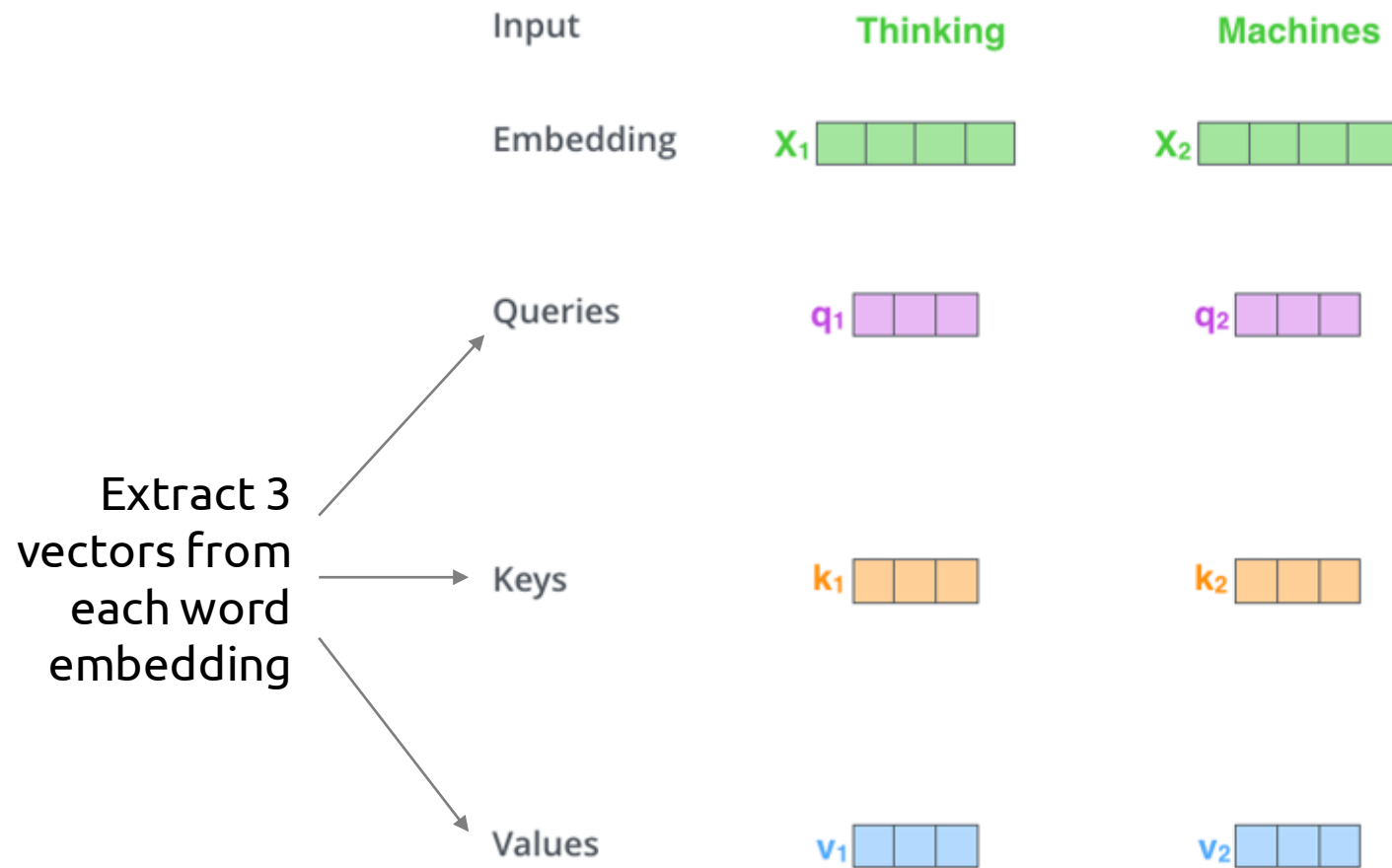
1. To determine how much attention a word should pay to each other other, we compute a **query vector** for the word and compare it to a **key vector** for every other word... **to compute our alignment score**
2. To produce the output vector, we sum up the **value vectors** for each word, weighted by the score we computed in step 1



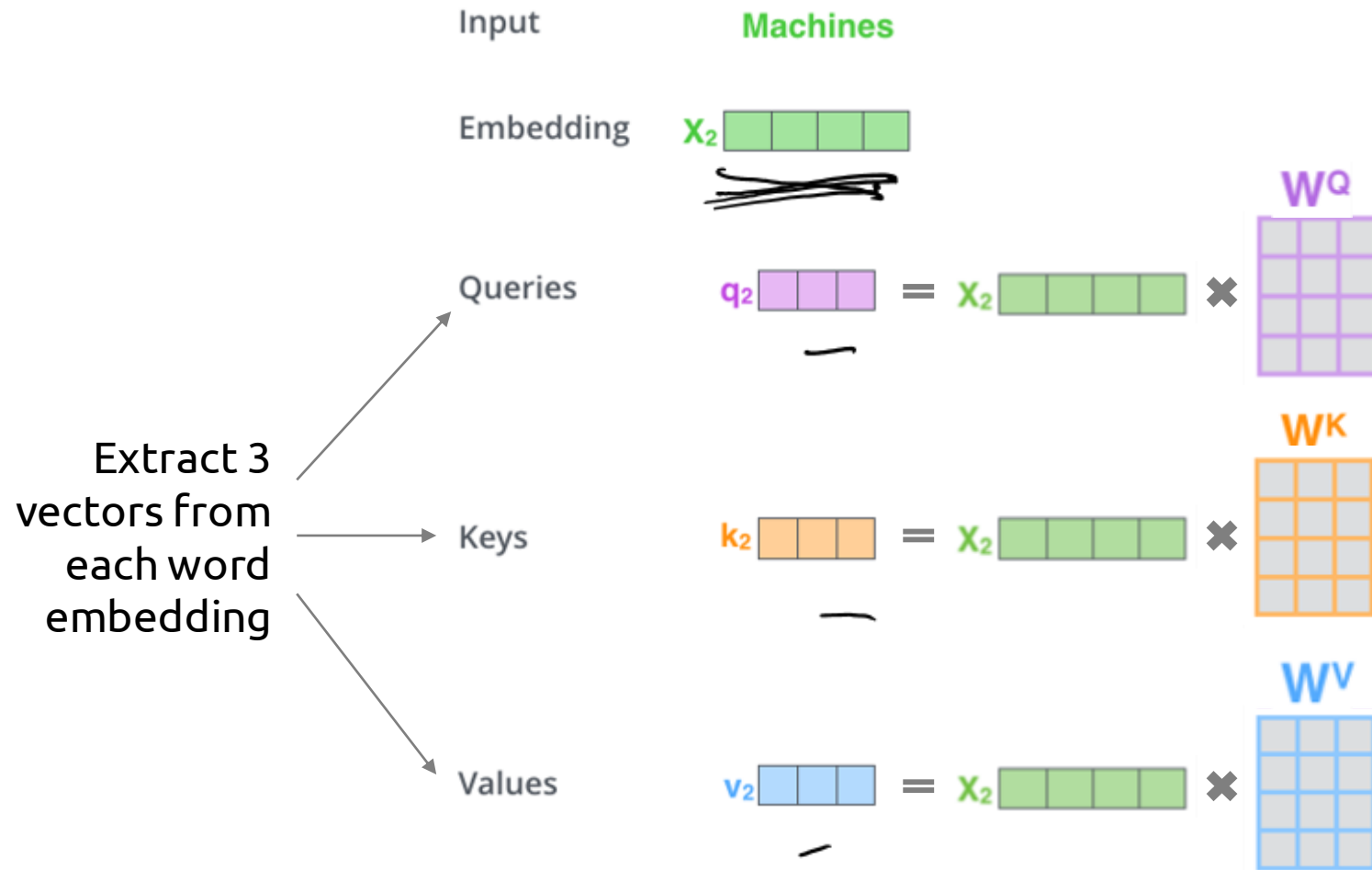
Self-Attention: Details



Self-Attention: Details



Self-Attention: Details



Each vector is obtained by multiplying the embedding with the respective weight matrix.

How do we get these weight matrices?

These matrices are the ***trainable parameters*** of the network

Self-Attention: Details

Computing self-attention for “Thinking”

What do we
calculate next?

Input

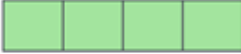
Embedding

Queries

Keys

Values

Thinking

x_1 

q_1 

,

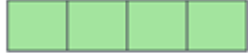
k_1 

,

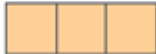
v_1 

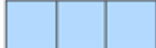
,

Machines

x_2 

q_2 

k_2 

v_2 

Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,\dots,n}$).

Input

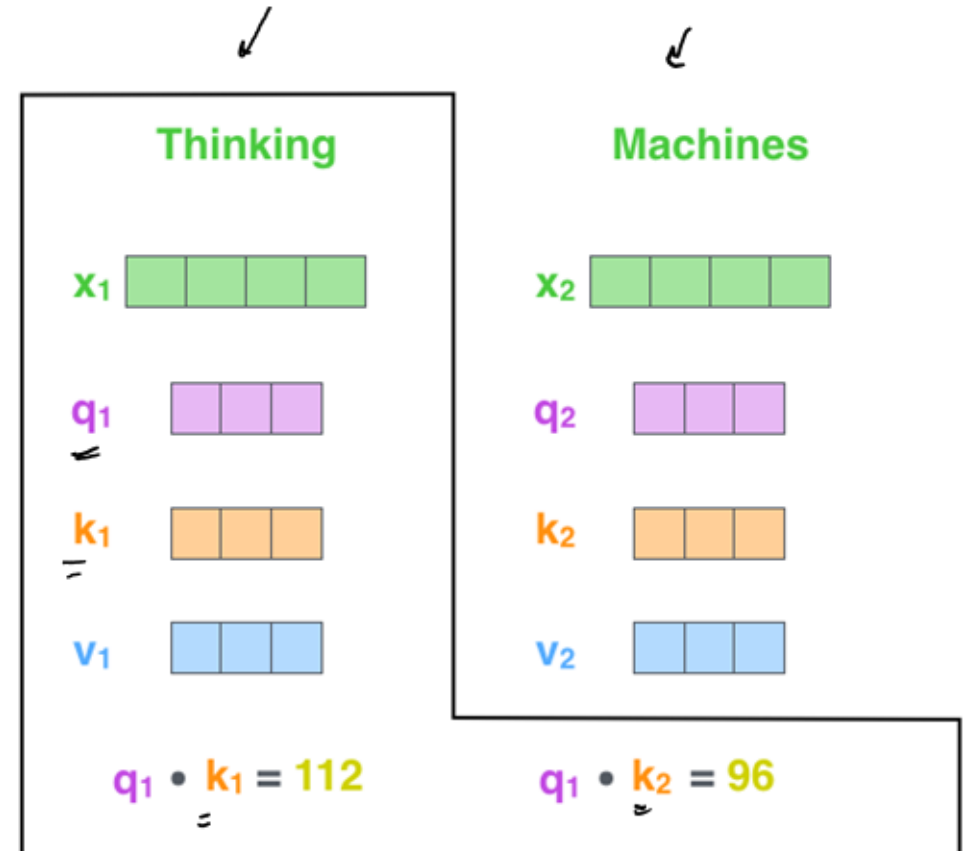
Embedding

Queries

Keys

Values

Score



Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,\dots,n}$).

What this is essentially asking is:
How much should “Thinking” pay attention to each other word in the sequence?

Query vectors are asking the question and key vectors respond.

Input

Embedding

Queries

Keys

Values

Score

Thinking

x_1 [] [] [] []

q_1 [] [] []

k_1 [] [] []

v_1 [] [] []

$$q_1 \cdot k_1 = 112$$

Machines

x_2 [] [] [] []

q_2 [] [] []

k_2 [] [] []

v_2 [] [] []

$$q_1 \cdot k_2 = 96$$

What do we calculate next?

Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,\dots,n}$).
2. **Scale:** Divide each score by square root of key vector dimensionality. Results in more stable gradients.

Input

Embedding

Queries

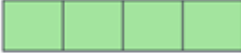
Keys

Values

Score

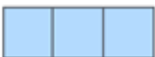
Divide by 8 ($\sqrt{d_k}$)

Thinking

x_1 

q_1 

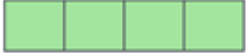
k_1 

v_1 

$q_1 \cdot k_1 = 112$

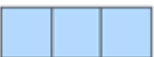
14

Machines

x_2 

q_2 

k_2 

v_2 

$q_1 \cdot k_2 = 96$

12

Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,\dots,n}$).
2. **Scale:** Divide each score by square root of key vector dimensionality. Results in more stable gradients.
3. **Softmax:** Apply softmax layer.

Input

Embedding

Queries

Keys

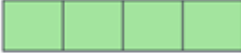
Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Thinking

x_1 

q_1 

k_1 

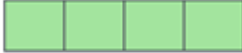
v_1 

$q_1 \cdot k_1 = 112$

14

0.88

Machines

x_2 

q_2 

k_2 

v_2 

$q_1 \cdot k_2 = 96$

12

0.12

Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,\dots,n}$).
2. **Scale:** Divide each score by square root of key vector dimensionality. Results in more stable gradients.
3. **Softmax:** Apply softmax layer.
4. **Weighting:** Multiply value vector of each word in the sentence ($v_{1,2,\dots,n}$) with the respective softmax values.

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

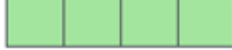
Softmax

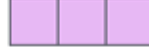
Softmax

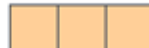
X

Value

Thinking

x_1 

q_1 

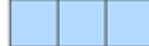
k_1 

v_1 

$q_1 \cdot k_1 = 112$

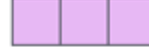
14

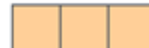
0.88

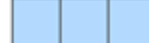
v_1 

Machines

x_2 

q_2 

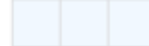
k_2 

v_2 

$q_1 \cdot k_2 = 96$

12

0.12

v_2 

Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,...,n}$).
2. **Scale:** Divide each score by square root of key vector dimensionality. Results in more stable gradients.
3. **Softmax:** Apply softmax layer.
4. **Weighting:** Multiply value vector of each word in the sentence ($v_{1,2,...,n}$) with the respective softmax values.

The idea here is that the value vectors store the contextual information that each word provides.

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

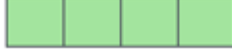
Softmax

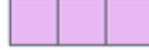
Softmax

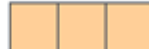
X

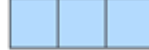
Value

Thinking

x_1 

q_1 

k_1 

v_1 

$q_1 \cdot k_1 = 112$

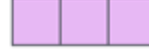
14

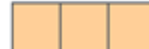
0.88

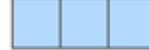
v_1 

Machines

x_2 

q_2 

k_2 

v_2 

$q_1 \cdot k_2 = 96$

12

0.12

v_2 

Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,\dots,n}$).
2. **Scale:** Divide each score by square root of key vector dimensionality. Results in more stable gradients.
3. **Softmax:** Apply softmax layer.
4. **Weighting:** Multiply value vector of each word in the sentence ($v_{1,2,\dots,n}$) with the respective softmax values.
5. **Sum:** Sum up weighted value vectors ($v_{1,2,\dots,n}$) into one final self-attention vector for “Thinking” (z_1)

Input

Embedding

Queries

Keys

Values

Score

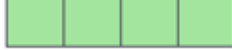
Divide by 8 ($\sqrt{d_k}$)

Softmax

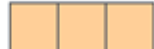
Softmax
X
Value

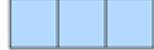
Sum

Thinking

x_1 

q_1 

k_1 

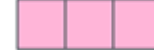
v_1 

$q_1 \cdot k_1 = 112$

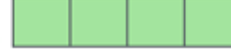
14

0.88

v_1 

z_1 

Machines

x_2 

q_2 

k_2 

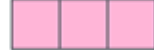
v_2 

$q_1 \cdot k_2 = 96$

12

0.12

v_2 

z_2 

Self-Attention: Details

Computing self-attention for “Thinking”

1. **Score:** Dot product query vector for “Thinking” (q_1) with the key vectors of each word in the sentence ($k_{1,2,...,n}$).
2. **Scale:** Divide each score by square root of key vector dimensionality. Results in more stable gradients.
3. **Softmax:** Apply softmax layer.
4. **Weighting:** Multiply value vector of each word in the sentence ($v_{1,2,...,n}$) with the respective softmax values.
5. **Sum:** Sum up weighted value vectors ($v_{1,2,...,n}$) into one final self-attention vector for “Thinking” (z_1)

We are weighting the *context* provided by each word by the amount of *attention* we should pay.

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

Thinking

x_1

q_1

k_1

v_1

$q_1 \cdot k_1 = 112$

14

0.88

v_1

z_1

Machines

x_2

q_2

k_2

v_2

$q_1 \cdot k_2 = 96$

12

0.12

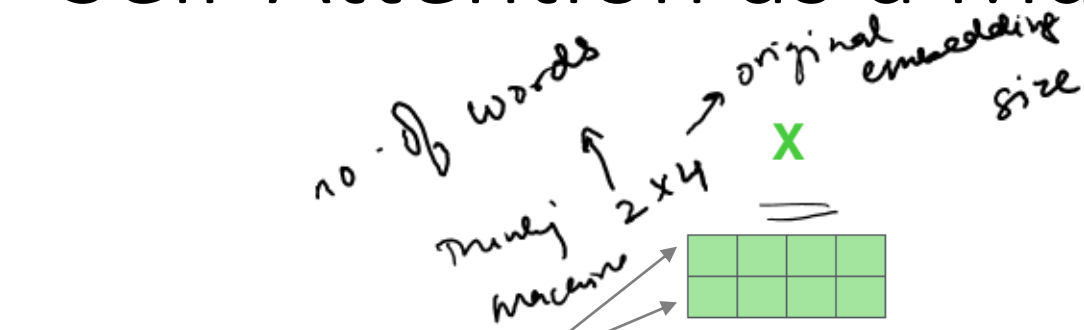
v_2

z_2

Any questions?

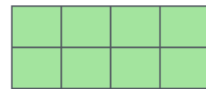


Self-Attention as a Matrix Computation

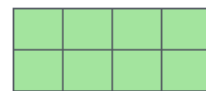


Each row of $\mathbf{X}$ is a word embedding of a word in our sentence.

$\mathbf{X}$



$\mathbf{X}$



Get your pens/papers or tablets ready!

What would be the dimensions of the weight matrices to calculate the query, key, and value?

What would be the dimensions of the query, key, and value matrices?

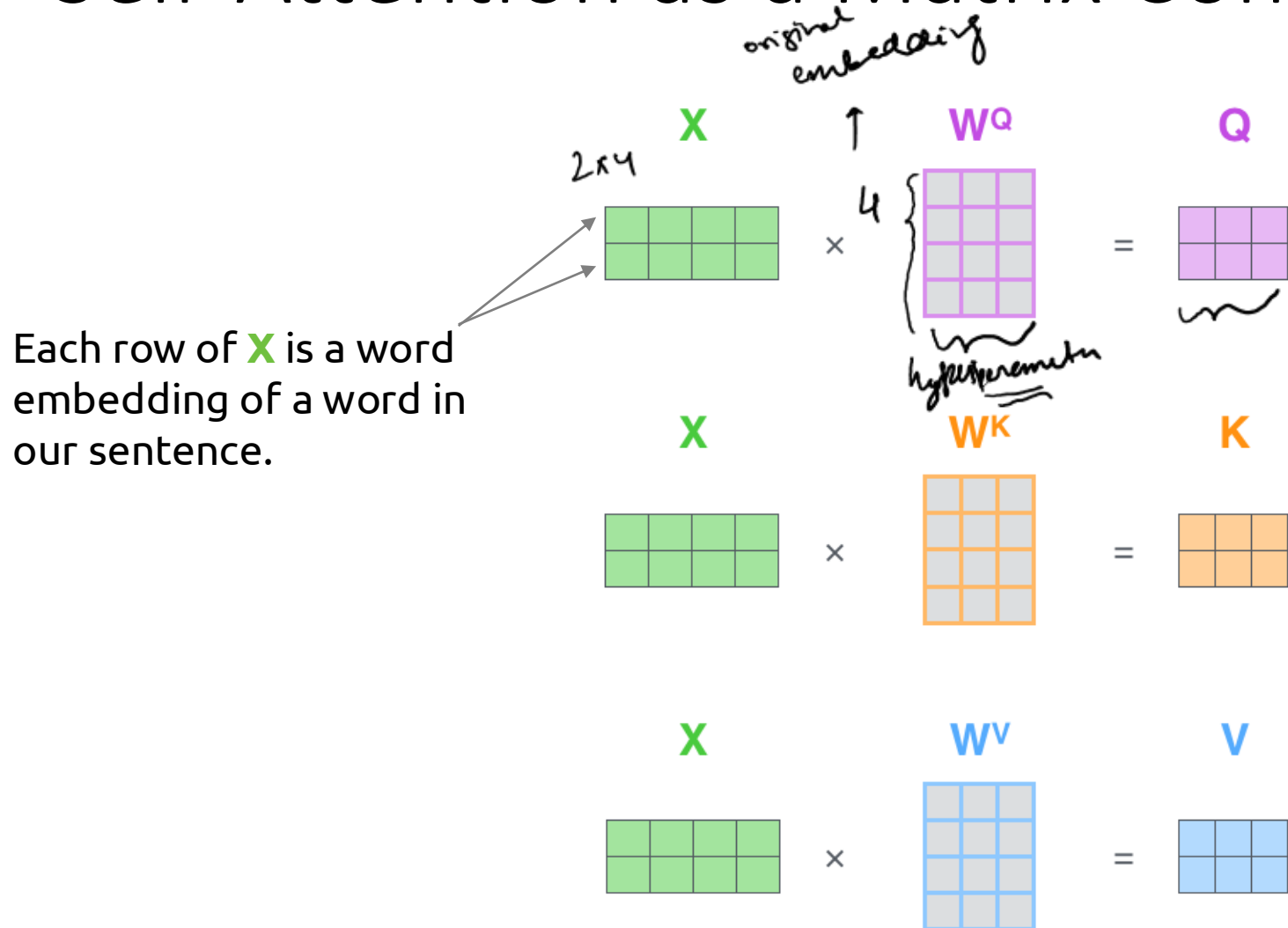
Apply the steps of calculating attention weights on the query and key matrices.

What is the dimension of attention weight matrix?

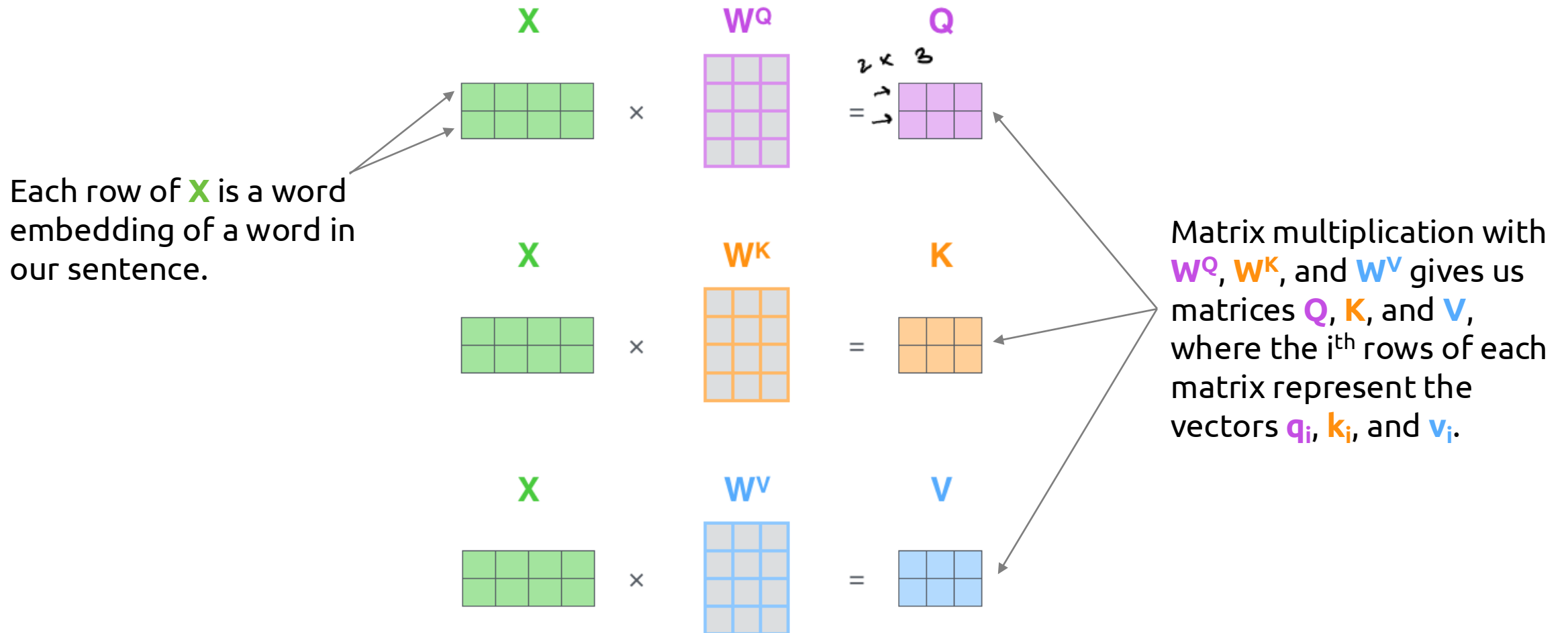
Multiply the attention weight matrix to value matrix produce the output matrix.

What are the dimensions of output matrix?

Self-Attention as a Matrix Computation



Self-Attention as a Matrix Computation



Self-Attention as a Matrix Computation

Matrix multiplying **Q** and the transpose of **K** calculates all the "score" values.

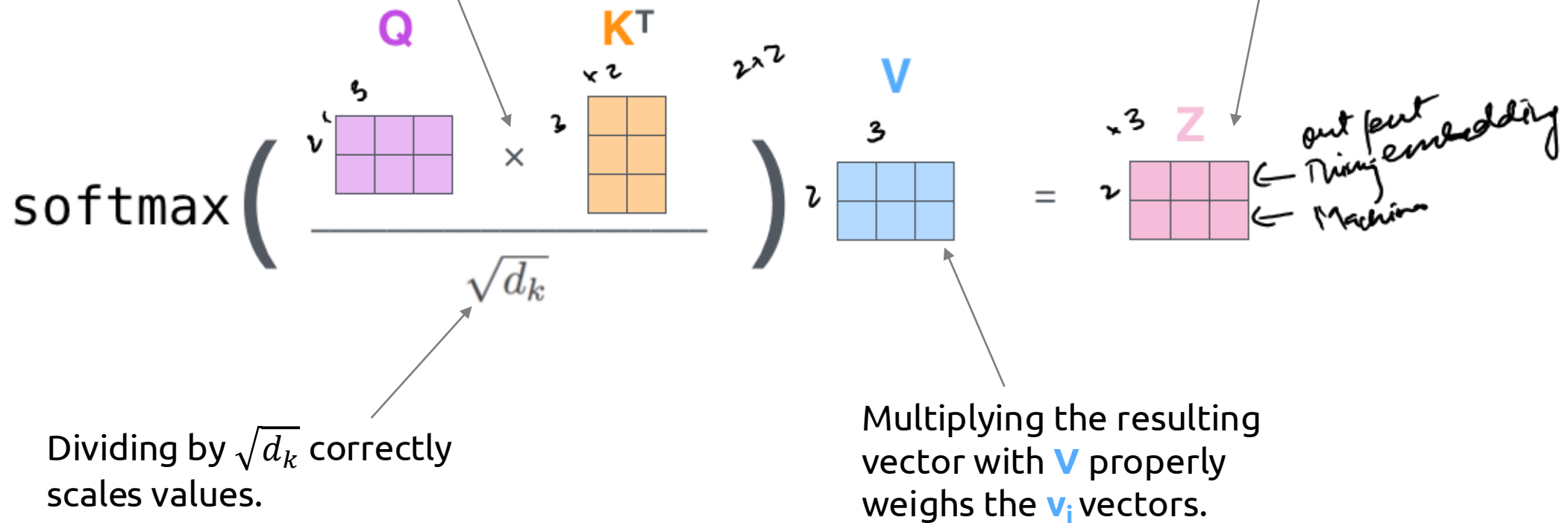
The diagram illustrates the matrix multiplication of **Q** and **K^T**. Matrix **Q** is a 2x3 grid of purple squares, with a handwritten '3' below it indicating the number of words. Matrix **K^T** is a 3x2 grid of orange squares, with a handwritten '2' below it indicating the number of words. An arrow labeled 'w' points to the multiplication symbol 'x' between the two matrices. The result of the multiplication is shown as a 2x2 grid of squares, with a handwritten 'no. of words' above it. The entire expression is enclosed in large parentheses, with a horizontal line separating the product from the denominator $\sqrt{d_k}$.

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ 3 \end{matrix} \times \begin{matrix} \text{K}^T \\ 2 \end{matrix}}{\sqrt{d_k}} \right) = \begin{matrix} \text{no. of words} \\ 2 \times 2^T \end{matrix}$$

Dividing by $\sqrt{d_k}$ correctly scales values.

Self-Attention as a Matrix Computation

Matrix multiplying **Q** and the transpose of **K** calculates all the "score" values.

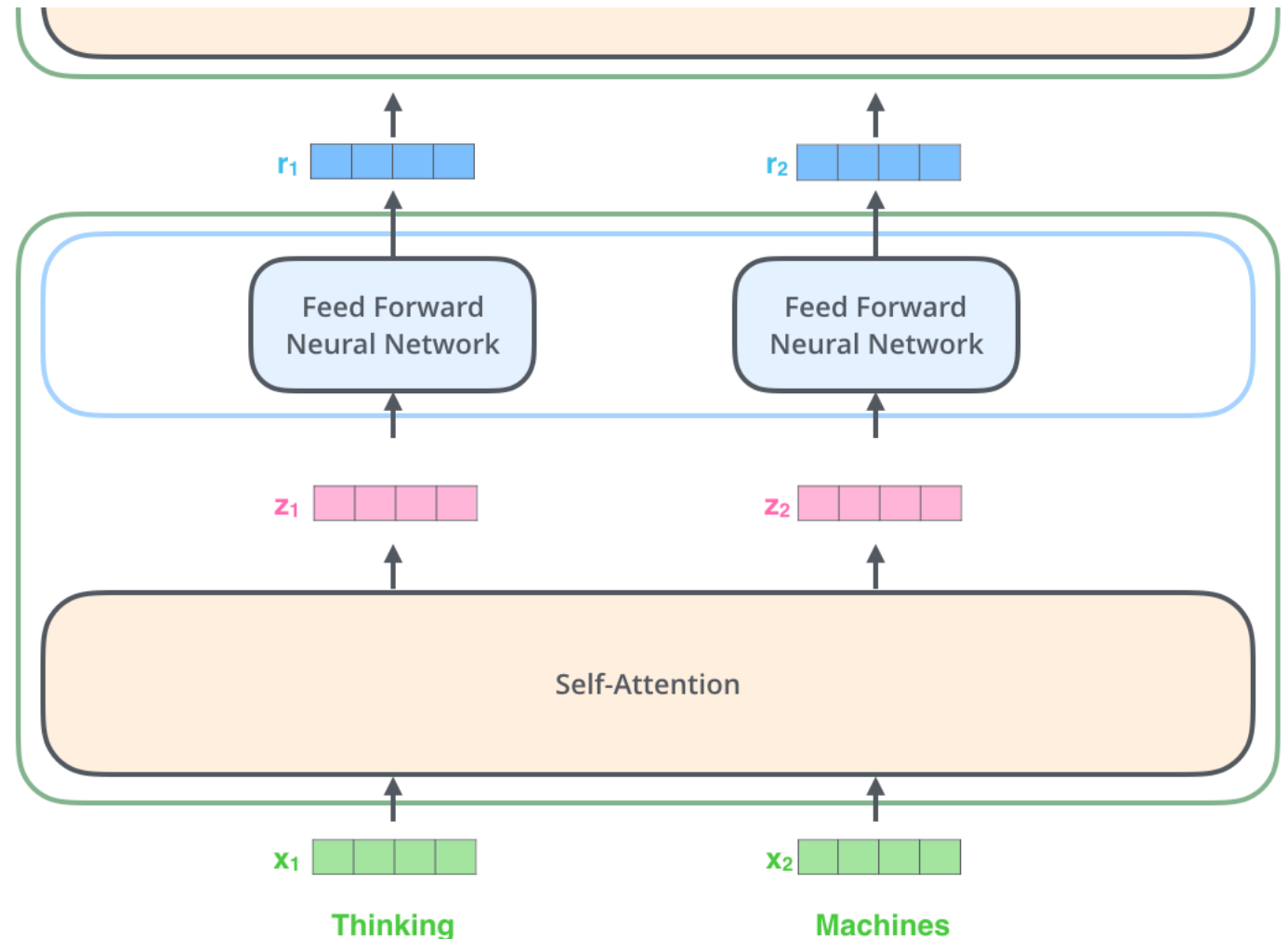


Any questions?



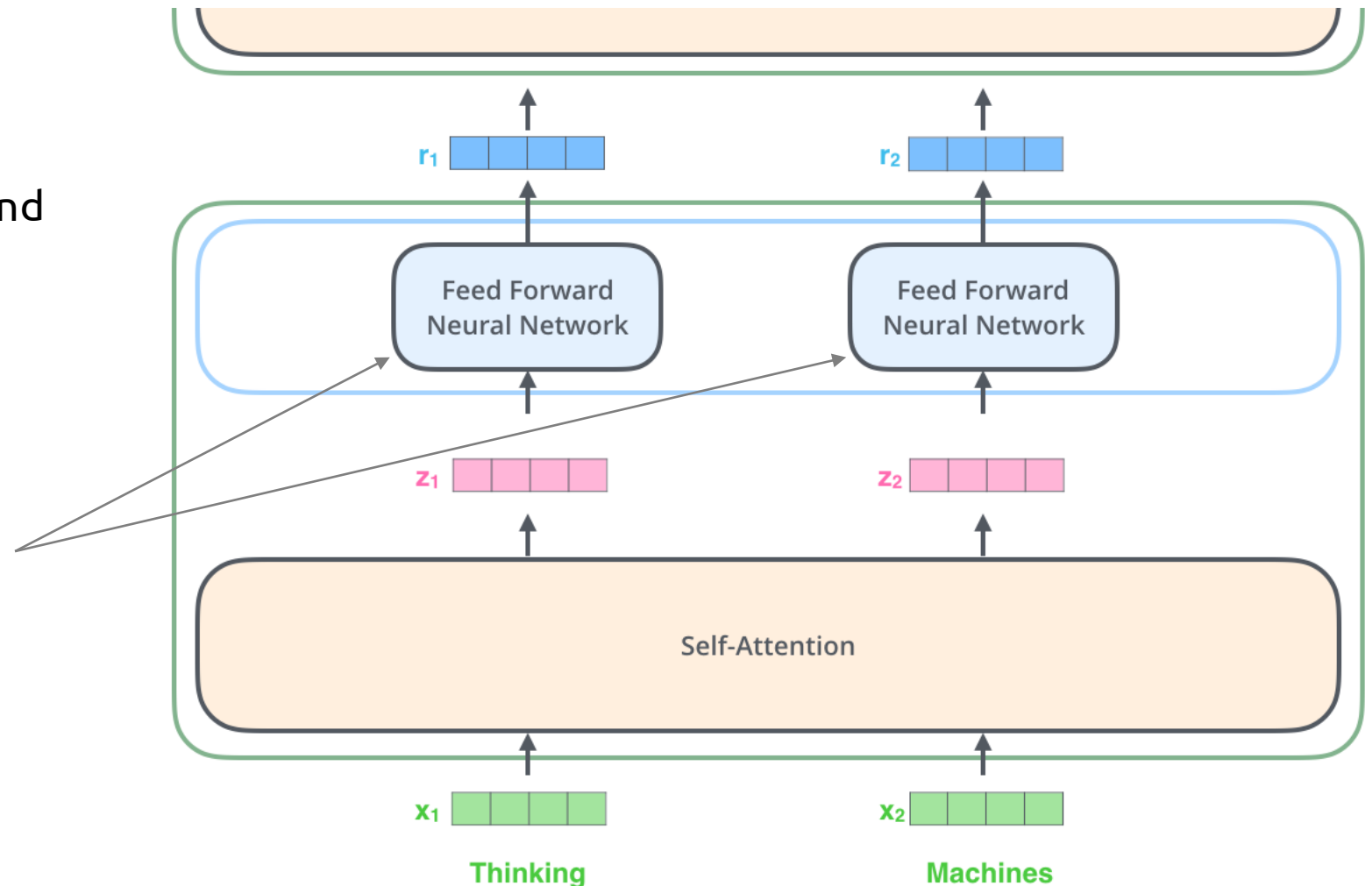
Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.
- Self-Attention layer is applied to each word individually.



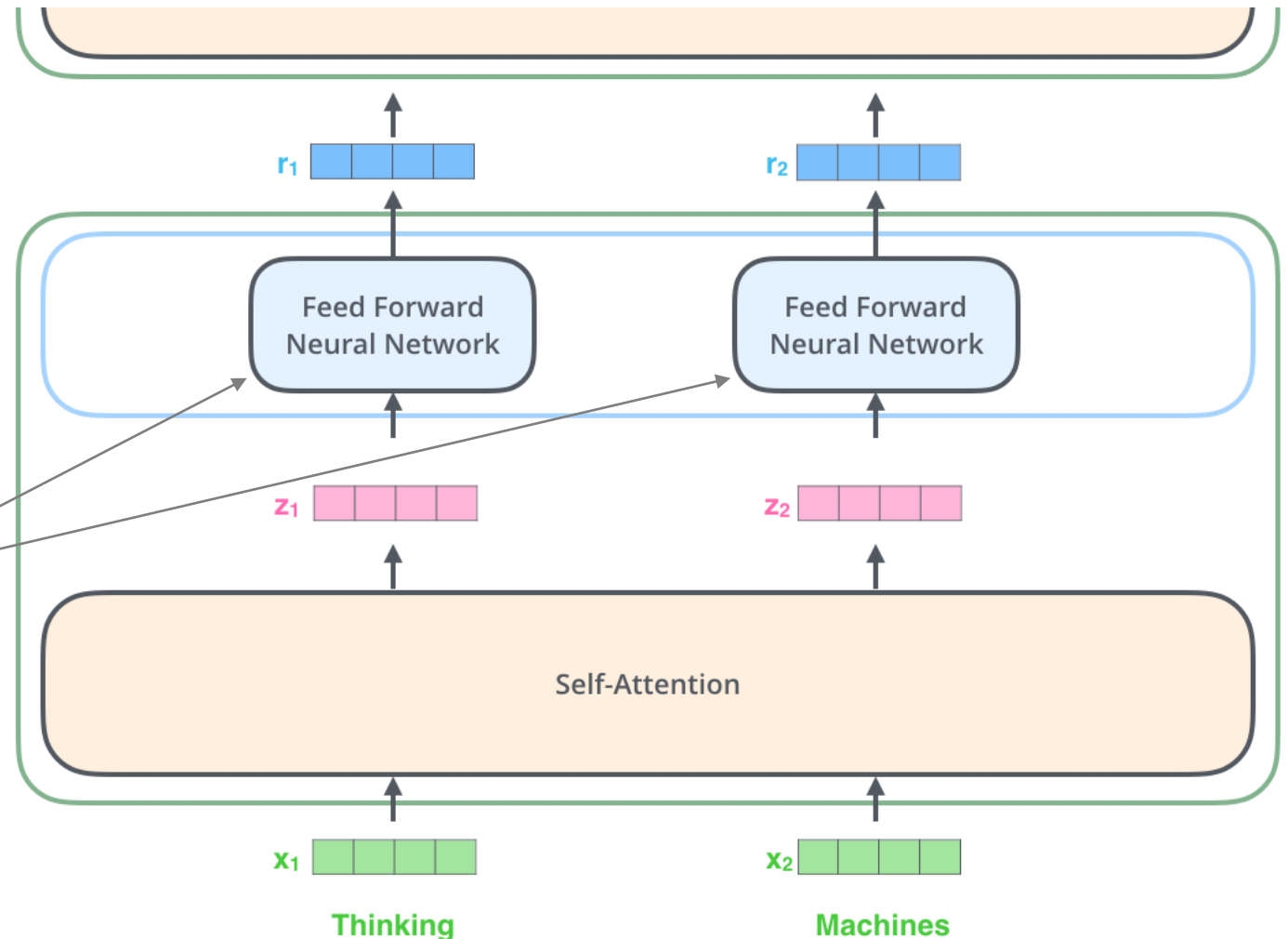
Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.
- Self-Attention layer is applied to each word individually.
- Feed Forward layer is applied to each word individually.



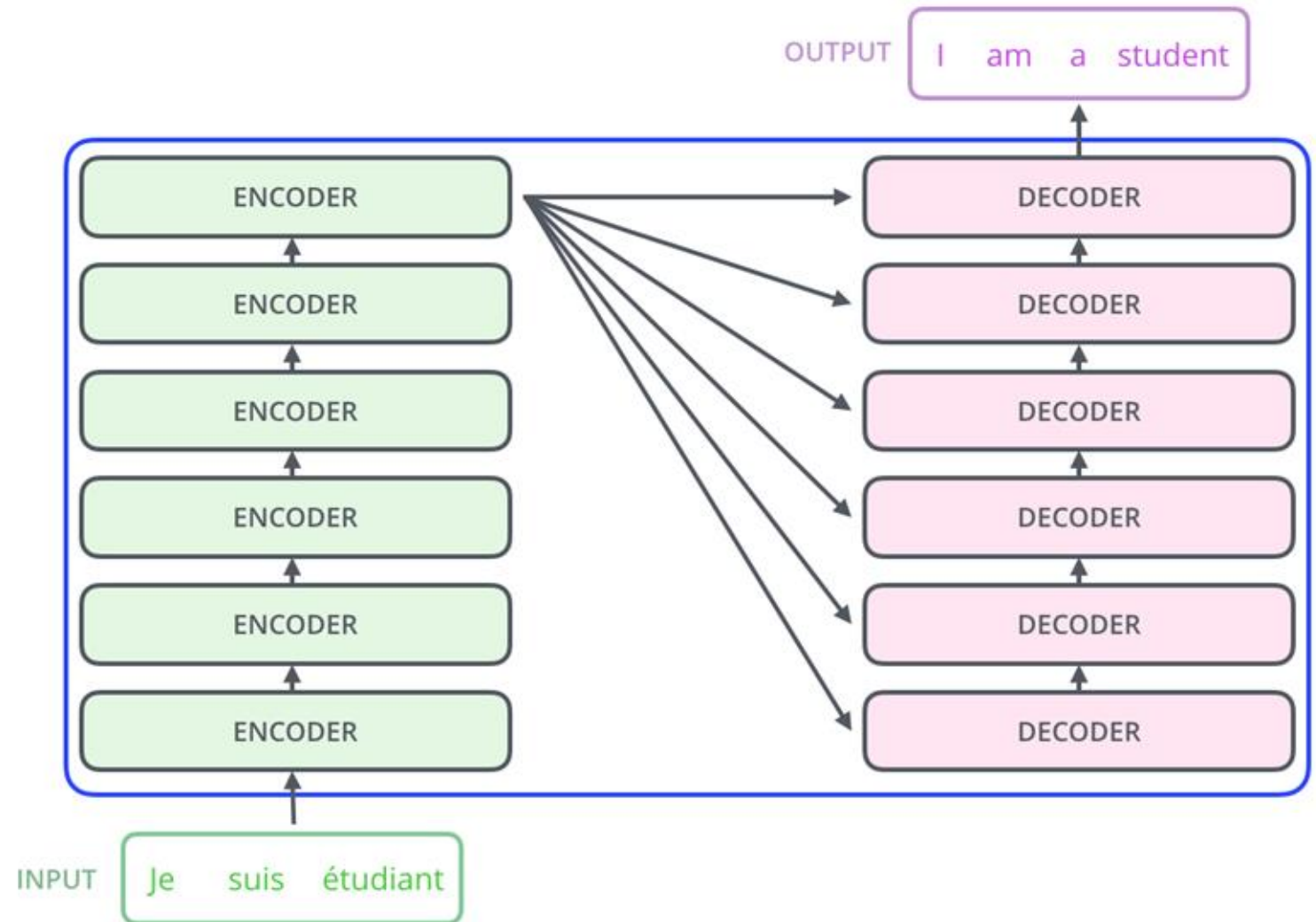
Encoder Block Map

- Encoder block breaks down into two main parts: Self-Attention, and Feed Forward layers.
- Self-Attention layer is applied to each word individually.
- Feed Forward layer is applied to each word individually.
- The outputs of the feed forward layer are then passed as the inputs of the next encoder block.



More to come on Transformer!

- **Positional encodings**
- **Multi-headed attention**
- **Modifications for efficiency**
- **Decoder**



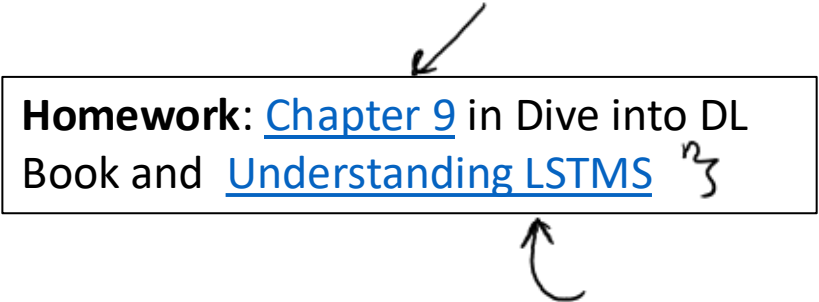
Recap – learn about RNNs and Transformers

(1) Recurrent Neural Network (RNN)

(2) Sequence-to-sequence modeling

(3) Transformers

(4) Class activity: Self-attention as matrix operation



Homework: [Chapter 9](#) in Dive into DL Book and [Understanding LSTMS](#) }

Wrap up



What was the clearest point today?

What was the muddiest point today?

